

Document Image Classification Using Graph Neural Networks

Executive Summary

The Challenge: Classifying scanned document images is a foundational task in enterprise workflows, yet it remains challenging because conventional CNN pipelines treat documents as flat pixel grids and ignore the spatial layout cues, such as header–body–footer structure, tabular arrangements, and marginal notes, that humans rely on to distinguish one document type from another. This limitation is most visible on structurally similar classes such as memo versus letter or form versus questionnaire, where CNN-only classifiers tend to underperform.

Our Approach: We develop a number of Graph Neural Network (GNN) based pipelines for document image classification that explicitly captures spatial layout structure and compare their performance against ResNet-50. The ResNet-50 model is pre-trained and fine-tuned on the grayscale RVL-CDIP dataset to bridge the domain gap from natural photographs. Its convolutional feature maps are then converted into graphs in which patches become nodes and spatial proximity (k-nearest neighbours) defines the edges. Three GNN variants are GCN, GAT, and GraphSAGE, and are implemented as graph-level classifiers and compared against the CNN-only baseline. Ablation studies on graph connectivity (k) and network depth isolate the contribution of graph-based reasoning.

Key Findings: The three architectures jointly reveal a clean trade-off: the GNN's in-distribution cost relative to the CNN is roughly proportional to its out-of-distribution gain. The pattern suggests that the more a GNN's graph formulation departs from the CNN's native representation geometry, the less faithful it is to the CNN on clean data but the more robust it becomes under distribution shift, because the reshaping is structurally orthogonal to whatever the CNN encoded.

Report Structure: Section 1 frames the problem and surveys related work. Section 2 describes the RVL-CDIP dataset and our exploratory analysis. Section 3 presents Part A, a ResNet-50 baseline pre-trained with transfer learning by gradually unfreezing increasing numbers of hidden layers. Section 4 presents Part B, which detail our experiments with various GNN models and their performance against the baseline. Section 5 documents the deployment of the end-to-end application.

Source Code: <https://github.com/sagerstack/deep-learning-doc-classification>

Table of Contents

1. Background and Introduction	3
1.1 Problem Statement and Motivation	3
1.2 Related Works	3
1.3 Problem Formulation and Solution Overview	3
2. Dataset	4
2.1 RVL-CDIP Overview	4
2.2 Experimental Dataset	4
2.3 Data Exploration	4
3. PHASE A: Baseline ResNet-50 System	5
3.1 Transfer Learning	5
3.2 Baseline Feature Extraction	6
3.3 CNN Variant on 100k Subset	7
3.4 Phase A Summary and Conclusion	7
4. PHASE B: GNN Model Classifiers	8
4.1 Experimental Summary	8
4.2 Graph Attention Network (GAT)	10
4.3 GraphSAGE	14
4.4 Graph Convolution Network (GCN)	19
4.5 Conclusion	22
5. Document Classification Research App	24
5.1 Application Overview	25
5.2 Source Code / Steps to Reproduce Models and Plots	25
5.3 Application Setup & Deployment Guide	26
5.4 Application Architecture	27
5.5 Development Methodology	28
5.6 Model Versioning	29
5.7 Model Performance Monitoring	29
5.8 Observability	30
6. Conclusion	31

1. Background and Introduction

1.1 Problem Statement and Motivation

Documents are a fundamental part of enterprise workflows yet classifying them automatically from scanned images remains difficult. Traditional CNN-based approaches treat document images as flat pixel grids and ignore the spatial structure and layout patterns that humans exploit to tell apart a form, an invoice, a memo, or a scientific report.

This project aims to develop a Graph Neural Network (GNN) based classifier that explicitly encodes spatial layout. By converting CNN feature maps into graphs (patches as nodes, spatial proximity as edges) we explore whether GNNs can learn structural regularities such as header–body–footer layouts, tabular arrangements, and margin patterns that complement or outperform purely convolutional baselines.

The primary challenges we address are: (1) fine-tuning a pretrained CNN on grayscale document images that differ significantly from the natural photographs it was trained on; (2) designing an effective graph construction strategy from CNN feature maps; (3) determining which GNN architecture (GCN, GAT, or GraphSAGE) best captures document structure; and (4) running ablation studies to investigate the performance of graph-based reasoning against CNNs.

1.2 Related Works

Harley et al. (2015) introduced the RVL-CDIP benchmark and the first CNN baselines (AlexNet, GoogLeNet) for document image classification. Kipf and Welling (2017) proposed Graph Convolutional Networks (GCNs), which generalise convolution to arbitrary graph structures via spectral-based message passing. Veličković et al. (2018) introduced Graph Attention Networks (GATs), in which learned attention coefficients weight neighbour contributions. Hamilton et al. (2017) presented GraphSAGE, an inductive framework for scalable representation learning over large graphs. More recently, GVDoc (Nguyen et al., 2023) applied graph-based reasoning to document understanding by constructing region graphs from detected layout elements. Our work has drawn inspiration from these research papers. More recently, Luo et al. (2022, Doc-GCN) applied heterogeneous GCNs to document layout analysis, demonstrating that graph-based reasoning can meaningfully improve downstream document understanding tasks.

1.3 Problem Formulation and Solution Overview

Problem Definition: Given a document image I , predict a class label $y \in \{\text{letter, memo, email, filefolder, resume, scientific publication, form, invoice, advertisement, budget, news article, presentation, questionnaire, specification, handwritten, scientific report}\}$ — 16 classes from the RVL-CDIP benchmark together with a confidence score.

Our Solution proceeds in two phases. In Phase A, we establish CNN baselines: a ResNet-50 fine-tuned on the full 400k RVL-CDIP dataset and a parallel baseline on a 100k stratified subset, providing the performance target that GNNs must match. In Phase B, we convert CNN feature maps into spatial graphs — nodes represent image patch regions, edges encode feature space k nearest neighbor relationships — and apply GNN architectures to classify documents by reasoning over these graphs. A fusion strategy combining global CNN features with GNN derived spatial features yields our best models. We further evaluate on RVL-CDIP-N, a dataset of 1,000 born digital color documents, to test whether graph based structural reasoning provides robustness advantages under distribution shift.

2. Dataset

2.1 RVL-CDIP Overview

We use the RVL-CDIP (Ryerson Vision Lab Complex Document Information Processing) dataset, the standard benchmark for document image classification. It contains 400,000 grayscale document images evenly distributed across 16 classes: letter, form, email, handwritten, advertisement, scientific report, scientific publication, specification, file folder, news article, budget, invoice, presentation, questionnaire, resume, and memo. The dataset then undergoes a pre-defined split of 320,000 / 40,000 / 40,000 for training, validation, and test respectively.

To assess generalisation to out-of-distribution, born-digital colour documents, we additionally evaluate on RVL-CDIP-N, a companion set of 1,000 modern colour documents spanning the same 16 classes. Both datasets are publicly available on HuggingFace.

Input: a document image (grayscale or colour), such as a scanned invoice, letter, or form.

Output: a predicted class label from the 16 document categories, together with a confidence score.

2.2 Experimental Dataset

Training GNN models on the full 400,000 image dataset requires significant compute per experiment. Feature extraction, graph construction, and message passing all add overhead beyond standard CNN training. To enable systematic experimentation across 47 configurations, we created a stratified 100k subset (80,000 train / 10,000 validation / 10,000 test), preserving the 16-class balance at 5,000 training images per class. We verified that a ResNet-50 fine-tuned on this subset achieves an accuracy of 0.8643, which is close to the full dataset result of 0.8947. This confirms the subset is representative and that conclusions drawn from it transfer to the full-scale setting.

2.3 Data Exploration

The RVL-CDIP dataset uses the 16 classes for image labels as shown in the figure below.

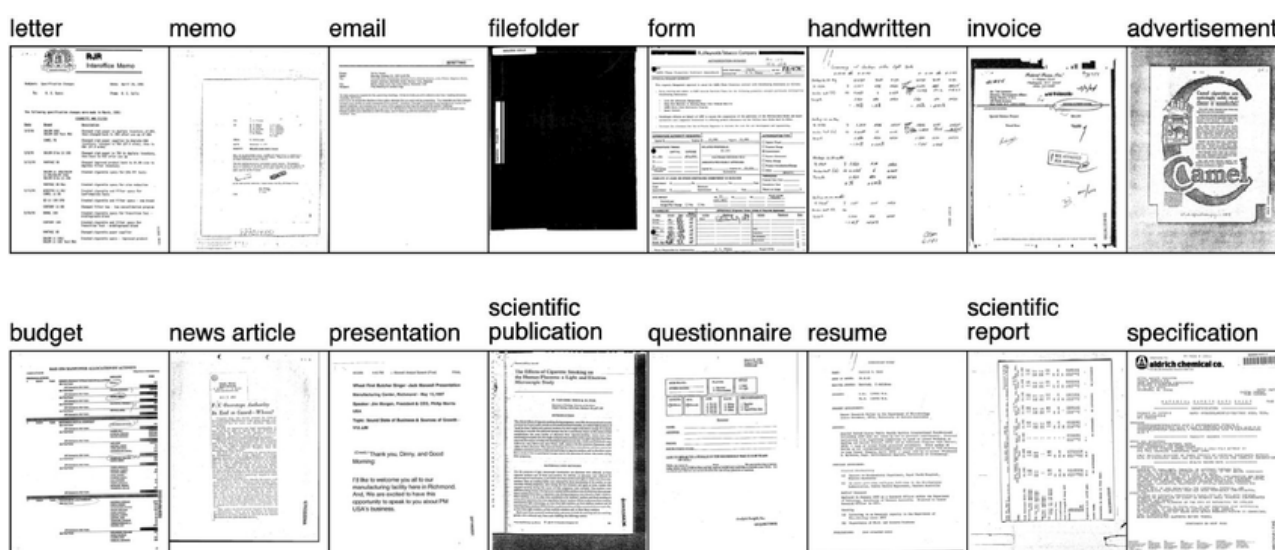


Figure 1: RVL-CDIP dataset sample images from each of the 16 classes

The labels are evenly spread among the 16 classes, with approximately 25,000 images per class. As can be observed in the sample images, the document images are grayscale and take the form of imperfect scans taken of the documents,

where there is a mix of text, images, figures, tables, etc., in a variety of layouts such as prose, columnar, structured form, etc.

3. PHASE A: Baseline ResNet-50 System

Phase A establishes a CNN-only baseline for document image classification using Transfer Learning that was covered in class. This baseline provides the reference against which the Phase B GNN-based experimental performance is measured.

3.1 Transfer Learning

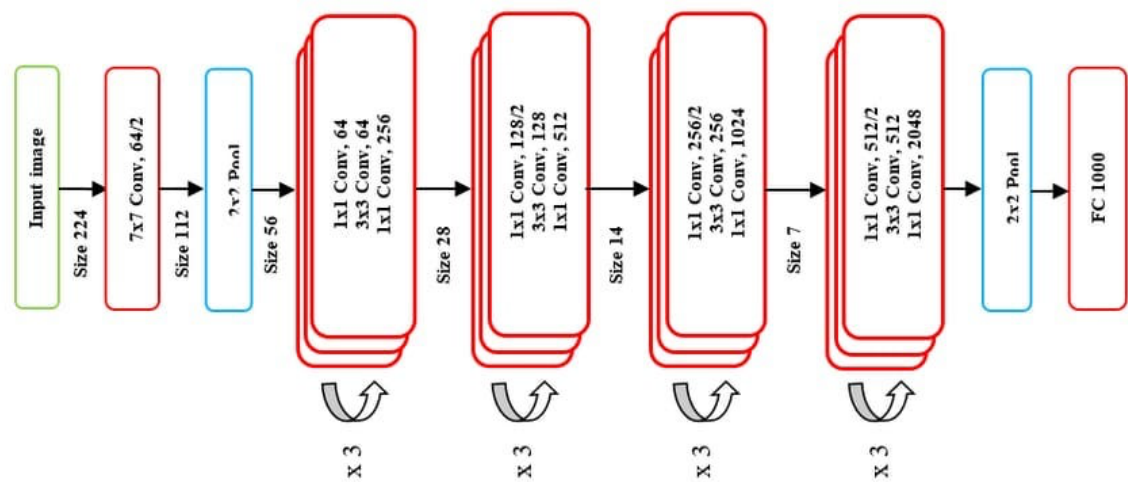


Figure 2: ResNet-50 architecture (<https://medium.com/data-science/the-annotated-resnet-50-a6c536034758>)

layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
conv2_x	56×56	3×3 max pool, stride 2				
		$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$
conv3_x	28×28	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 8$
conv4_x	14×14	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 23$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 36$
conv5_x	7×7	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$
	1×1	average pool, 1000-d fc, softmax				

Figure 3: <https://medium.com/data-science/the-annotated-resnet-50-a6c536034758>

ResNet-50 takes an input image of size 224 x 224 x 3 and passes it through a 50-layer deep convolutional neural network consisting of 48 convolutional layers, 1 MaxPool layer, and 1 AveragePool layer. ResNet-50 was originally trained on the ImageNet database and outputs labels from 1000 object categories. To implement transfer learning for the RVL-CDIP dataset on ResNet-50, certain changes needed to be made to the original ResNet-50 model. As the RVL-CDIP dataset contains images from 16 classes,

The first change would be to resize the input images from RVL-CDIP into 224 x 224, and converting the single channel grayscale image into 3-channel RGB images. Next, we swap out the final 1000-dimension fully connected layer of ResNet-50 for a 16-dimension fully connected layer so that it fits the number of classes in the RVL-CDIP dataset.

Transfer learning is performed on the ResNet-50 model (pre-trained on ImageNet) in 3 stages of gradual unfreezing of layers to train with the full RVL-CDIP dataset with a split of 320,000 / 40,000 / 40,000 for training, validation, and test respectively. The tables below summarises the settings used at each stage and the training accuracy results.

Stage	Trainable params	Frozen params	Optimizer (param groups)	LR schedule	Init from	Checkpoint
FC head only	fc (~33K, <0.2%)	conv1 + bn1 + layer1 + layer2 + layer3 + layer4	Adam: fc @ 1e-3	StepLR(step=3, $\gamma=0.1$), 10 epochs	ImageNet-1K (IMAGENET1K_V2)	best_model_resnet50_01.pt
layer4 + FC	layer4 + fc (~15M, ~64%)	conv1 + bn1 + layer1 + layer2 + layer3	Adam: layer4 @ 1e-4, fc @ 1e-3	StepLR(step=3, $\gamma=0.1$), 10 epochs	best_model_resnet50_01.pt	best_model_resnet50_02.pt
layer3 + layer4 + FC	layer3 + layer4 + fc (~22M, ~94%)	conv1 + bn1 + layer1 + layer2	Adam: layer3 @ 1e-5, layer4 @ 1e-4, fc @ 1e-3	StepLR(step=3, $\gamma=0.1$), 10 epochs	best_model_resnet50_02.pt	best_model_resnet50_03.pt

Table 1: Summary of Transfer Learning Stages applied to ResNet-50

Stage	Trainable params	Final train acc	Final train loss	Best val acc	Test acc	Notes
FC head only	32,784 (~0.14%)	0.6783	1.0697	0.6883 (epoch 10)	0.6858	Linear-probing ceiling on frozen ImageNet features
layer4 + FC	~15M (~64%)	0.9797	0.0720	0.8893 (epoch 10)	(not evaluated)	Largest single jump: +20.1 pp val acc over Stage 1
layer3 + layer4 + FC	~22M (~94%)	0.9972	0.0113	0.8951 (epoch 9)	0.8947	Diminishing return: only +0.6 pp val acc over Stage 2

Table 2: Summary of Transfer Learning results per stage

3.2 Baseline Feature Extraction

To make use of the image embeddings from the newly pre-trained ResNet-50 model, features can be extracted at different parts of the convolutional layer pipeline to form nodal features for the different GNN models used in this

project.

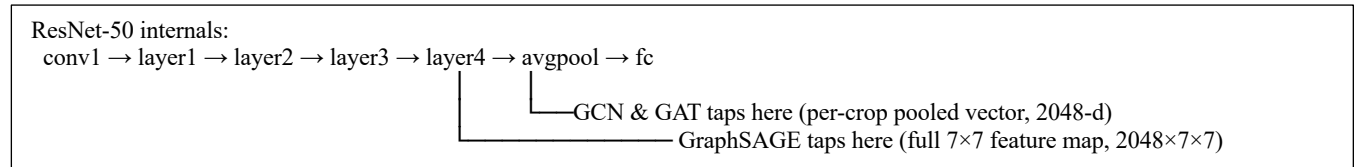


Figure 4: ResNet-50 Backbone used in GNN models

3.3 CNN Variant on 100k Subset

To support the volume of GNN experiments, we operate on the 100k stratified subset described in Section 2. A ResNet-50 is fine-tuned end to end on the 80,000 image training split, replicating the same training procedure as the full 400k model. The model achieves accuracy of 0.8643, and F1 of 0.8644, close to the full dataset result of 0.8947, confirming the subset is representative. This number is the performance target for all GNN experiments in Phase B.

3.4 Phase A Summary and Conclusion

Phase A establishes three conclusions that drive the remainder of the project:

1. Fine-tuned CNN is a strong, hard to beat baseline. At an accuracy of 0.8947, it sets a high performance benchmark that the GNN models must approach to be considered competitive.
2. Feature quality is foundational. Fine-tuning the backbone on domain-specific document images is non-negotiable. GNN experiments in Phase B inherit these fine-tuned representations as node features, ensuring all comparisons are made on equal footing.
3. The CNN baseline is not the full picture. A model that achieves 0.8947 accuracy on scanned grayscale documents from 1990s archives may differ significantly from modern born digital inputs. Phase A does not evaluate this.

4. PHASE B: GNN Model Classifiers

Phase B develops Graph Neural Network classifiers that reason over the spatial structure of document images, evaluating whether explicit graph-based reasoning can match or exceed the ResNet-50 baseline established in Phase A. Rather than treating a document as a flat pixel grid, a graph is constructed from either each image individually in which nodes represent spatial regions of the CNN feature map and edges encode relationships between those regions, or using the whole corpus to construct a graph where each node is one document image. Three GNN architectures are trained and compared: a Graph Convolutional Network (GCN), GraphSAGE, and a Graph Attention Network (GAT). Given the computational cost of training on the full 400,000-image dataset, a stratified 100k subset was used to systematically explore graph construction strategies, feature regimes, and architectural choices across more than 40 experiments. The findings from that search directly shaped some of the final models, which were trained end-to-end on the full dataset.

4.1 Experimental Summary

To enable systematic exploration, all architecture and configuration experiments were conducted on the stratified 100k subset described in Section 2.2. These experiments were run across graph construction strategies, feature regimes, regularisation configurations, and GNN architectures. The findings from this search directly drove the design of the three final models trained on the full dataset.

Exp	Configuration	Dataset	Accuracy	vs CNN	Observations	Outcome
—	ResNet-50 fine-tuned (CNN baseline)	100k	0.8643	—	Performance target for all GNN experiments	Reference
—	ResNet-50 fine-tuned (CNN baseline)	400k	0.8947	—	Performance target for final fine-tuned GNN models	Reference
0	Vanilla GraphSAGE, spatial grid k-NN (k=8), frozen features	2.5k	0.6156	−2.03%	GNN loses across all 16 classes; fixed graph topology provides no document-specific structure	Discard
3	Feature-space cosine k-NN (k=8), frozen features	2.5k	0.6219	−1.40%	Gap narrows; form +7.2%, handwritten +5.6% vs CNN; document-specific topology helps structured classes	Keep
11	Feature-kNN GraphSAGE, scaled to 10k	10k	0.6160	+0.80%	GNN wins overall for the first time; winning class profile shifts to complex-structured documents	Keep
12	Regularised feature-kNN (dropout, DropEdge, cosine LR)	10k	0.6620	+5.62% F1	2% threshold met on full 16-class task; regularisation was the bottleneck, not architecture	Keep
14b	Fine-tuned ResNet-50 CNN baseline established	100k	—	—	Fine-tuned CNN (0.8643) dominates frozen-feature GNN by 10.14%; proper baseline confirmed	Reference
15	Regularised GraphSAGE on fine-tuned features	100k	0.8533	−1.10%	GNN loses all 16 classes to a linear classifier on the same features; message-passing blurs optimised representations	Discard
16	Fusion GraphSAGE: concat(global_feat, GNN output)	100k	0.8636	−0.07%	Matches CNN; preserving the global feature via fusion sets a floor equal to the linear baseline	Keep

19	End-to-end fine-tuning through fusion GNN	100k	0.8573	−0.70%	Backbone co-adaptation degrades fine-tuned representations; frozen features with fusion is superior	Discard
20	Ensemble: fine-tuned CNN + fusion GNN	100k	0.8649	+0.06%	Models agree on 95.9% of samples; too similar for ensemble to exploit complementary signal	Discard
21	Fusion GraphSAGE OOD evaluation on RVL-CDIP-N (1,002 samples)	100k	0.6427 OOD	+4.59% OOD	GNN wins 10/12 classes on born-digital documents; spatial structure transfers better than texture features	Keep
23	Architecture comparison: GCN vs GAT vs GraphSAGE (fusion)	100k	GAT 0.8605 GCN 0.8574 SAGE 0.8540	—	GAT best per epoch; SAGE best with longer training; architecture choice matters less than training duration	Keep
25–26	OCR Bag-of-Characters node features	100k	0.8549	−0.94%	Raw OCR concatenation adds noise on graphical and mixed-layout classes; peaks at epoch 1	Discard
45	DocumentGAT: YOLO regions + OCR + Sentence-BERT	100k	0.8645 val	+0.02% val	Multimodal node features push GAT past CNN on validation; richer node features compensate for structural limitations	Keep
47	GVDoc GraphSAGE: fusion + richer graph construction	100k	0.8656 val	+0.13% val	Best 100k result across all configurations	Keep
48	Transductive GCN baseline (k=10, 2-layer)	100k	0.8981	+3.38%	Initial global graph setup showed high apparent accuracy but revealed transductive leakage from test nodes shaping training representations.	Refine
49	GCN Sparsity Ablation (k=5 vs k=15)	100k	0.9106 (k=5)	+4.63%	Sparser graphs (k=5) significantly outperformed denser neighbourhoods (k=15), proving that excess edges introduce cross-class noise.	Keep (k=5)
50	GCN Depth Ablation (2 vs 3 layers)	100k	0.8919 (3-layer)	+2.76%	3-layer message passing consistently underperformed the 2-layer counterpart across all 100k values due to over-smoothing.	Discard 3-layer

Table 3 Summary of Graph Experiments and ablations conducted on 100k dataset

Three architectural principles emerged from this search that directly shaped the design of all three final models.

- **Message passing degrades fine-tuned representations.** Applying GraphSAGE directly on fine-tuned ResNet-50 features (Exp 15) produced the clearest negative result of the search: the GNN lost to a simple linear classifier on the same features by 0.75 percentage points, failing on all 16 classes. Neighbourhood aggregation averages features across spatial regions, blurring the discriminative structure that fine-tuning had encoded. The same pattern surfaced in text feature experiments (Exp 25-26): raw Bag-of-Characters concatenation degraded graphical classes by up to 24.5%, and a gated fusion variant learned nothing — its sigmoid gate collapsed to a constant 0.5. Both failures pointed to the same conclusion: appending noisy features or overwriting fine-tuned ones both degrade performance. The GNN needed to complement the CNN, not replace it.

- **Fusion architecture unlocks OOD robustness.** Concatenating the original global ResNet-50 feature vector with the GNN embedding before classification (Exp 16) resolved the degradation problem by preserving the full CNN representation as a shortcut. Fusion matched the fine-tuned CNN baseline to within 0.07% in-distribution. The more consequential finding came from OOD evaluation on RVL-CDIP-N (Exp 21): on 1,002 born-digital colour documents, the fusion GNN outperformed the CNN by +4.59% accuracy, winning 10 of 12 classes. CNN accuracy degraded 26.8% from in-distribution; GNN degraded only 22.1%. Document layout structure — encoded in the graph — transfers across the scanned-to-born-digital domain shift far more robustly than the texture features the CNN had learned from grayscale training data. This OOD finding became the primary justification for pursuing the GNN approach to final model training.

- **Overcoming the feature-grid ceiling requires richer node representation.** The fixed 7x7 CNN feature grid imposes a hard ceiling: every document produces the same 49-node topology and the GNN can only rearrange information the CNN already extracted. An architecture screening of ten GVdoc-inspired variants (Exp 28-37) confirmed this — the best 100k in-distribution result was 0.8575, remaining 0.68% below the CNN baseline regardless of fusion strategy. Critically, the 10k OOD rankings inverted at 100k scale: the cross-attention variant that led OOD on 10k (+3.99%) became the worst OOD model on 100k (-0.40%), while the simpler Exp 16 concat fusion remained the most robust. Overcoming the ceiling required document-specific graphs with semantically meaningful node features. This drove the shift to YOLO-detected regions and Sentence-BERT text embeddings for DocumentGAT — where word-level semantic content combined with spatial layout gave the GAT the inter-node relationships necessary to reason about document structure beyond what visual features alone could capture.

- **Cross-document consensus requires strict graph sparsity and gradual feature compression:** While GraphSAGE and GAT explored intra-document layouts, the GCN experiments (Exp 48–50) constructed a global k-NN graph treating entire documents as nodes. This search revealed three critical dynamics for inter-document reasoning. First, evaluating nodes present during graph construction artificially inflates accuracy via transductive leakage (Exp 48), establishing the need for strict inductive splits to measure true generalization. Second, the GCN is highly sensitive to graph sparsity: expanding the neighborhood (e.g., $k=15$) introduces cross-class noise, while keeping the graph sparse ($k=5$) forces the network to rely only on the highest-confidence visual matches. Finally, severe information bottlenecks (e.g., compressing directly from 2048 to 128 dimensions) stunt the network's capacity to learn, while adding a third message-passing layer consistently induces over-smoothing (Exp 50). Resolving this required a shallow, 2-layer architecture with a gradual bottleneck compression path to preserve discriminative signal.

These three findings directly drove the design of the final models trained on the full 400,000-image dataset. FusionGraphSAGE (Section 4.3) applies the Exp 16 architecture end-to-end, establishing the simplest proven GNN baseline. The GCN (Section 4.4) tests a transductive formulation in which the entire 320k training split forms a single graph, exploring whether global structural context changes the competitive dynamic. DocumentGAT (Section 4.2) takes the furthest departure from the CNN-feature baseline, replacing the fixed grid with YOLO-detected document regions and Sentence-BERT node embeddings, the architecture the experimental search identified as the path most likely to exceed the CNN in-distribution.

4.2 Graph Attention Network (GAT)

A Graph Attention Network (GAT) is a type of GNN that uses attention mechanisms to aggregate information from a node's directly adjacent neighbourhood (Velickovic et al., 2018).

Graph Features

The GAT uses a undirected graph whose nodes correspond to a layout region object in the document image. Each document image is decomposed into semantic regions using a YOLO-based layout detector, and each region becomes a node in a graph with four types of features:

1. Visual features (2048-dim) from a frozen ResNet-50 backbone (He et al., 2016)
2. Spatial features (4-dim) encoding normalized bounding box position and size
3. Layout class (11-dim) one-hot encoding of the region's semantic type (text, table, figure, etc.) from DocLayNet (Pfitzmann et al., 2022)
4. Text content (384-dim) sentence embedding of OCR-extracted text via Sentence-BERT (Reimers & Gurevych, 2019)

A global context node carrying the full-image feature is connected to all region nodes, forming a hub-and-spoke graph structure. The GAT learns to classify documents by attending over the spatial and semantic relationships between regions.

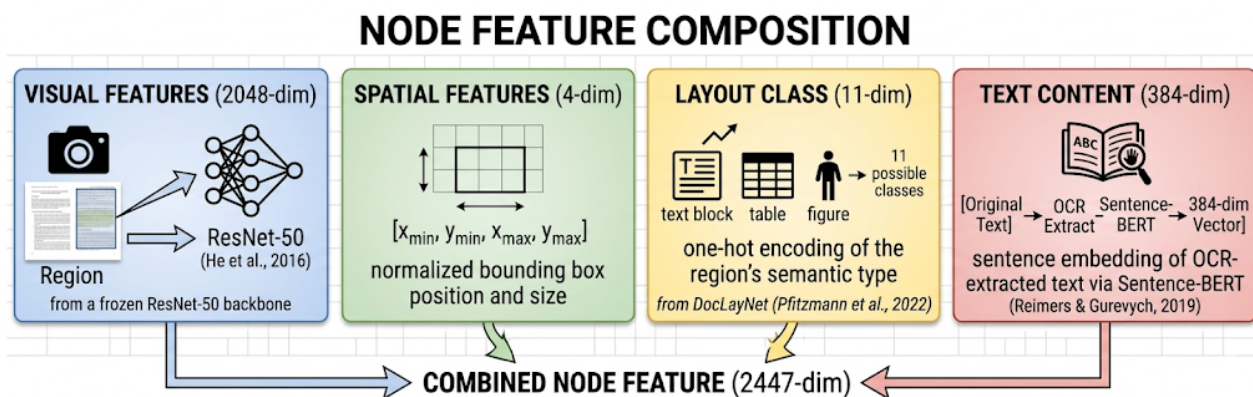


Figure 5: GAT node features

Semantic region detection is performed using a YOLOv8 model fine-tuned on DocLayNet (Pfitzmann et al., 2022), which identifies 11 categories of document elements: Caption, Footnote, Formula, List-item, Page-footer, Page-header, Picture, Section-header, Table, Text, and Title. Each detected region becomes a node in the graph.

Each detected region is cropped from the document image, resized to 224x224, and passed through the frozen ResNet-50 that we have fine-tuned on RVL-CDIP. The average-pooled output produces a 2048-dimensional feature vector per region. A global context node is also created by passing the entire document image through the same extractor, providing the GAT with a holistic document representation.

Text content is extracted from each region crop using Tesseract OCR and encoded into a 384-dimensional semantic embedding using all-MiniLM-L6-v2 (Reimers & Gurevych, 2019), a lightweight sentence transformer. Regions without detectable text (e.g. pictures, blank areas) receive a zero vector, providing a distinct signal that differentiates visual-only regions from text-bearing ones. The global node receives OCR text from the full page.

Edges are constructed using k-nearest neighbors ($k=8$) based on Euclidean distance between normalized region centroids. This captures spatial proximity, whereby nearby regions on the page are connected, allowing the GAT to learn layout patterns (e.g., a header above a body text block).

The global context node is appended to every graph, connected to all region nodes. This hub-and-spoke structure ensures every region can attend to the full-document representation in a single hop. The global node's bounding box is set to (0, 0, 1, 1) (full page) with an all-zeros layout class vector.



Figure 6: Example graph construction

Model Architecture

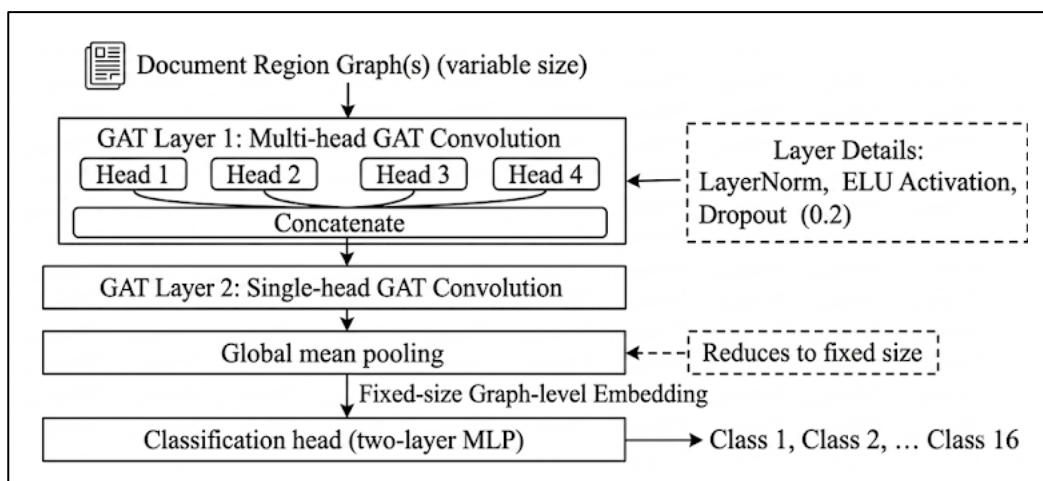


Figure 7: DocumentGAT model architecture

The model we created, DocumentGAT, uses the Graph Attention Network architecture to classify documents based on their region graphs. The architecture consists of:

1. Layer 1: Multi-head GAT convolution (4 heads, concat) — learns which neighboring regions are most relevant
2. Layer 2: Single-head GAT convolution — aggregates multi-head outputs into a unified representation
3. Global mean pooling — produces a fixed-size graph-level embedding from variable-size graphs
4. Classification head — two-layer MLP mapping to 16 document classes

Each GAT layer applies LayerNorm, exponential linear unit (ELU) activation, and a dropout of 0.2. The attention mechanism allows the model to learn which spatial and semantic relationships between document regions are most informative for classification.

Training Results

The model was trained with cross-entropy loss, Adam optimizer ($lr=1e-3$, weight decay= $1e-4$), and cosine annealing learning rate schedule. Gradient norms are clipped at 1.0 to prevent attention weight instabilities. Early stopping with patience=10 halts training when validation accuracy plateaus. Maximum epochs was set at 80.

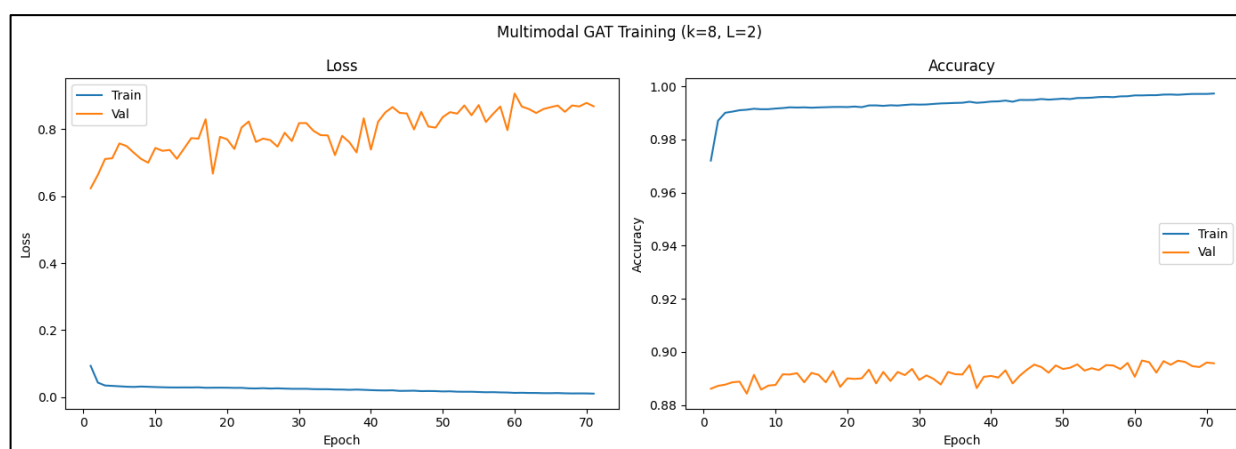


Figure 8: GAT Training/Validation Loss and Accuracy Plots

The best GAT trained was at epoch 61, with a validation accuracy of 0.8968. The in-distribution test accuracy for this model was 0.8955, and the out-of-distribution accuracy was 0.6577.

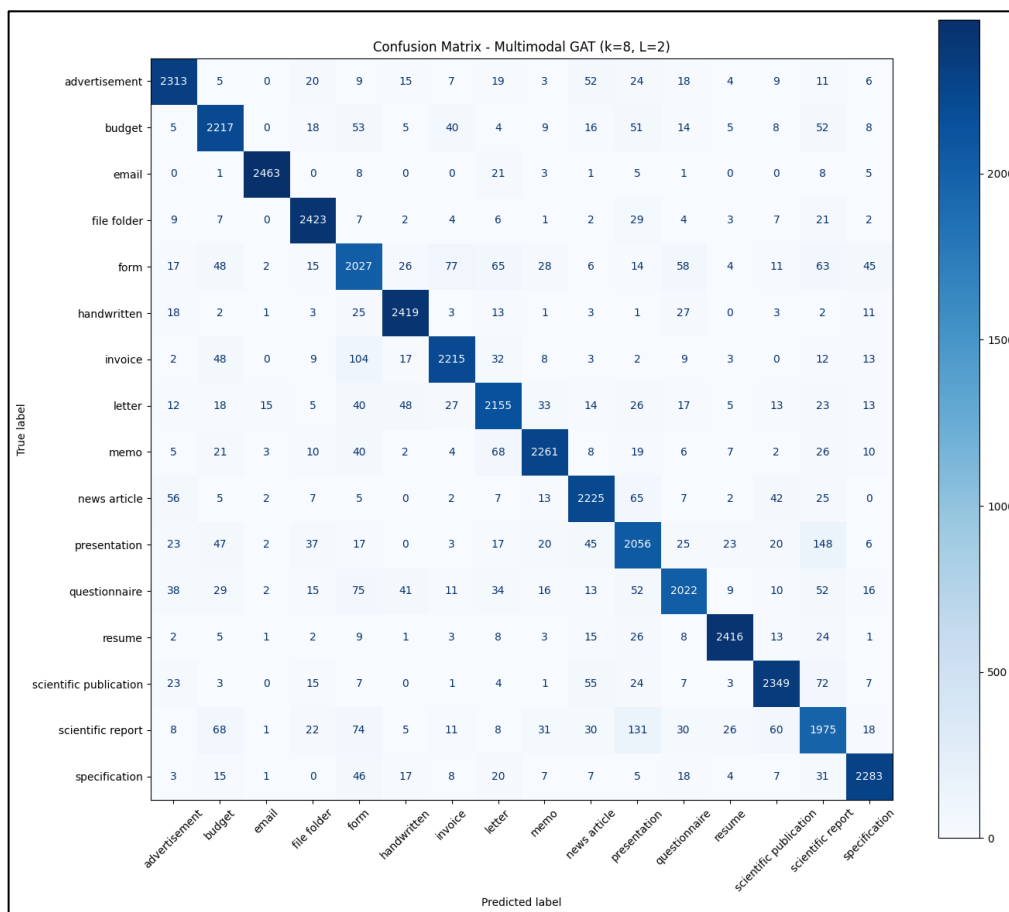


Figure 9: Confusion matrix of in-distribution test accuracy for best GAT model

Discussion

The GAT matches (acc=0.8955) but does not significantly exceed the ResNet-50 baseline (acc=0.8947). Across multiple graph construction strategies explored during this project (spatial grid (7x7), OCR-based regions, YOLO semantic regions, and multimodal features) the GAT consistently converged to ~89% validation accuracy. Adding OCR text embeddings provided a marginal improvement (+0.85% over vision-only GAT), but the graph structure itself does not contribute meaningful signal beyond what the CNN already captures from the full image.

The global context node, which carries the full-image ResNet feature, seems to dominate the attention mechanism during in-distribution testing. The GAT effectively learns to route classification decisions through this single node, reducing the graph-based model to a linear classifier on top of frozen CNN features, functionally equivalent to the ResNet baseline.

However, when an out-of-distribution test scenario is used, the GAT performs better than the ResNet baseline, with an accuracy of 0.6577 compared to ResNet's 0.6208. This suggests that the GAT model is more generalised to document images that were not part of the original dataset.

The GAT achieves comparable in-distribution accuracy and significantly better out-of-distribution accuracy with approximately 10x fewer trainable parameters (2.8M vs 23.5M). However, the full inference pipeline still requires the ResNet backbone for feature extraction, so deployment cost is not reduced.

4.3 GraphSAGE

GraphSAGE (Hamilton et al., 2017) is an inductive graph neural network framework that learns node representations by sampling and aggregating features from local neighbourhoods. Unlike transductive methods such as GCN, GraphSAGE generalises to new, unseen graphs at inference time — each document is processed as an independent

graph without requiring the full dataset to be present during training. This makes it well-suited to large-scale document classification where each image produces a distinct graph with its own topology. The final model, FusionGraphSAGE, incorporates the key architectural insight from the experimental search: a fusion shortcut that concatenates the global CNN representation with the GNN graph embedding before classification, preventing the representation degradation that arises when message passing is applied directly to fine-tuned features.

Graph Features

FusionGraphSAGE operates on a fixed-resolution graph derived directly from the ResNet-50 spatial feature map. Each document image is passed through the fine-tuned ResNet-50 backbone, and the layer4 output — a 7x7 spatial grid of 2048-dimensional feature vectors — forms the node set of the graph. This produces 49 nodes per document, each representing a roughly 32x32 pixel region of the original image. Each node carries a single feature type: a 2048-dimensional visual feature vector extracted directly from the ResNet-50 layer4 output for the corresponding spatial patch. No positional encoding is appended, spatial position is implicitly encoded through the content-driven k-NN edge topology rather than explicit coordinates.

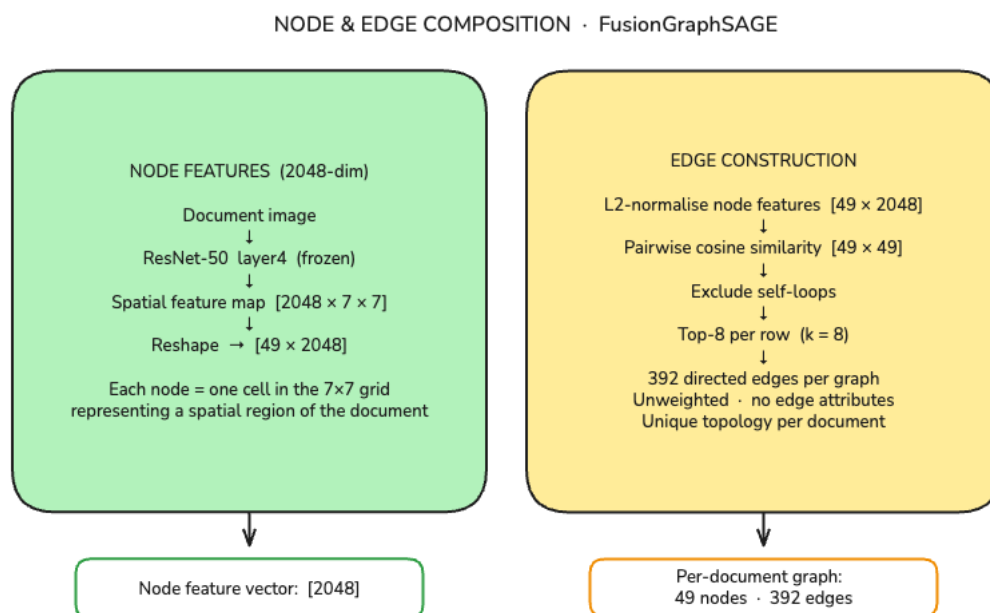


Figure 50: Fusion GraphSAGE Nodes and Edge features

Edges are constructed using feature-space cosine k-nearest neighbours ($k=8$): each node is connected to the 8 nodes whose ResNet feature vectors are most similar by cosine distance. This creates a document-specific graph topology in which patches with similar visual content are connected regardless of spatial position — text blocks connect to other text blocks, figures to other figures. This contrasts with a spatial grid where every document produces the same fixed topology regardless of content. A separate global feature vector (the ResNet-50 average-pool output, 2048-dim) is retained outside the graph for fusion with the GNN embedding at classification time.

FEATURE K-NN (K=8)

NON-LOCAL

FEATURE K-NN (K=8)

NON-LOCAL



Figure 61: FusionSAGE Graph construction with edges connecting spatial regions with similar visual appearance as encoded by the ResNet-50 backbone, rather than spatial proximity

Model Architecture

The FusionGraphSAGE architecture processes each document graph through two GraphSAGE convolution layers followed by a fusion-based classification head:

- Feature extraction — ResNet-50 backbone (frozen) processes the document image and produces two outputs: a conv5 spatial feature map reshaped to 49 nodes $\times$ 2048 features $[49 \times 2048]$, and a global average-pooled vector [2048]
- Graph construction — a content-driven k-NN graph ($k=8$, cosine similarity) is built over the 49 nodes using their 2048-dim features as embeddings, producing per-document edge topology with up to 392 directed edges
- SAGEConv Layer 1 ($2048 \rightarrow 256$, mean aggregator) — each node aggregates features from its 8 neighbours via mean pooling and projects to 256 dimensions; followed by ReLU activation and Dropout(0.5). Output: $[49 \times 256]$
- SAGEConv Layer 2 ($256 \rightarrow 128$, mean aggregator) — same aggregation pattern, projecting each node to 128 dimensions; followed by ReLU activation and Dropout(0.5). Output: $[49 \times 128]$
- Global mean pooling — averages all 49 node embeddings into a single graph-level representation. Output: [128]
- Fusion — the 128-dim graph embedding is concatenated with the 2048-dim global CNN feature to form a joint representation. Output: [2176]
- Classification head — two-layer MLP: Linear($2176 \rightarrow 512$) $\rightarrow$ ReLU $\rightarrow$ Dropout(0.5) $\rightarrow$ Linear($512 \rightarrow 16$), mapping to the 16 RVL-CDIP document classes

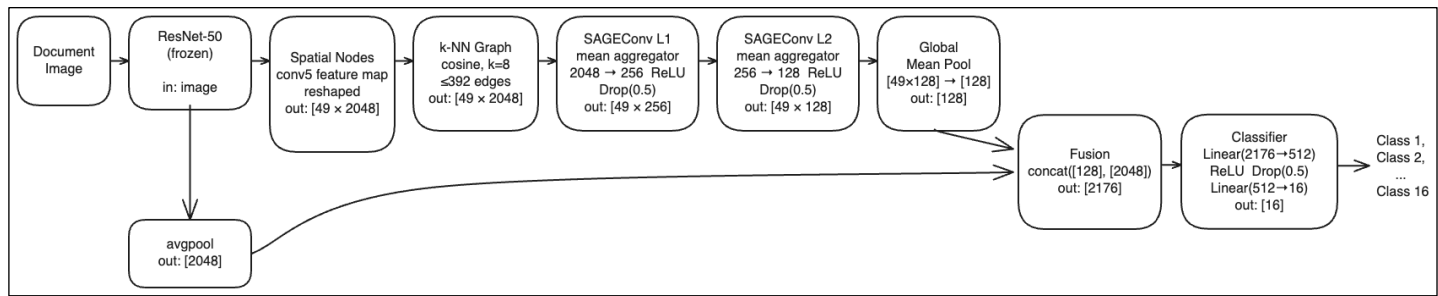


Figure 72: FusionSAGE Graph model architecture

The model has 2,237,328 trainable parameters. Regularisation during training includes DropEdge (20% of edges randomly removed per batch) and weight decay (5×10^{-4}). The fusion shortcut is the critical design choice: by preserving the full global CNN representation as a direct input to the classifier, the model cannot perform worse than a linear classifier on those features, regardless of the quality of the GNN output.

Training Results

FusionGraphSAGE was trained on the full 400,000-image RVL-CDIP dataset (320,000 train / 40,000 validation / 40,000 test). The Adam optimizer was used with an initial learning rate of 1×10^{-3} and cosine annealing to 1×10^{-5} . Early stopping with patience of 10 epochs on validation F1 was applied. Training ran for 20 epochs before early stopping triggered, with the best checkpoint at epoch 10 (val acc=0.8862, val F1=0.8862). Total training time was 165.5 minutes.

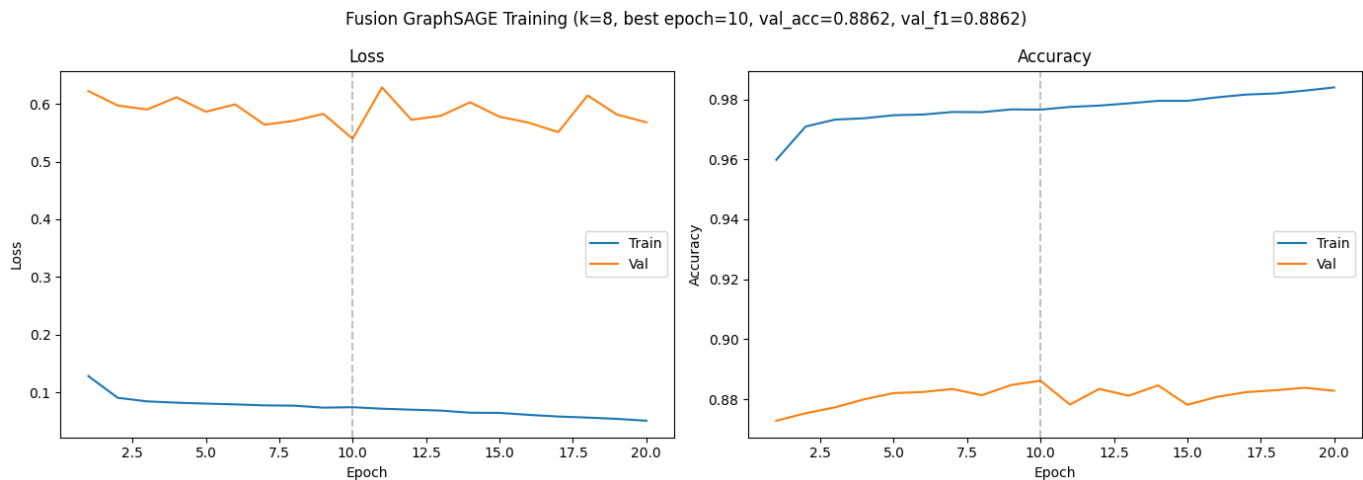


Figure 13: FusionGraphSAGE training and validation accuracy/loss curves

The model achieved an in-distribution test accuracy of 0.8847 (macro F1: 0.8843) on the RVL-CDIP 40k test set, compared to the ResNet-50 CNN baseline of 0.8947 — a gap of 1.00 percentage point. On the out-of-distribution RVL-CDIP-N evaluation set (1,002 born-digital colour documents, 12 classes), the model achieved an accuracy of 0.6707 (macro F1: 0.6687), compared to the CNN OOD baseline of 0.6208 — a +4.99 percentage point advantage.

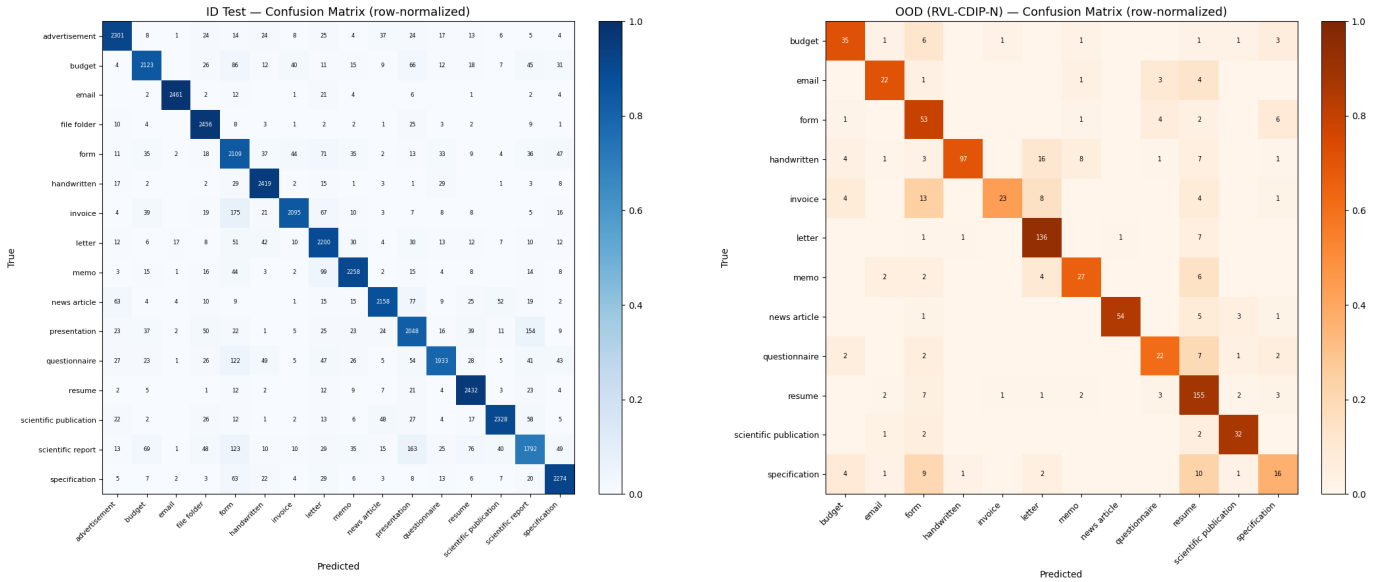


Figure 14: FusionGraphSAGE Confusion Matrix for RVL-CDIP (In-Distribution) and RVL-CDIP-N (Out of Distribution) datasets

Per-class performance on the in-distribution test set shows a clear split along document structure. The model performs strongest on classes with consistent, distinctive layouts: email (F1=0.983), handwritten (F1=0.934), and file_folder (F1=0.933). These classes carry predictable spatial regularities — header-body structure, consistent handwriting layout, and distinctive folder-tab patterns — that the feature-space k-NN graph reliably captures. Performance is weakest on visually heterogeneous classes: scientific_report (F1=0.757), form (F1=0.782), and presentation (F1=0.807). Scientific reports vary widely in layout between journal and technical formats; forms differ substantially across document issuers; presentation slides have inconsistent structure across decks.

Discussion

FusionGraphSAGE establishes the GNN baseline on the full dataset. The 1.00 percentage point gap to the CNN in-distribution reflects a known limitation of the architecture: the 7×7 feature grid is a coarse graph representation — 49 nodes with fixed spatial resolution — and the GNN can only rearrange information that the CNN feature extractor has already encoded. The experimental search on 100k (Section 4.1) confirmed this ceiling: no fusion variant across 47 experiments exceeded the CNN baseline on in-distribution data at scale. On the full 400k dataset, the CNN baseline achieves 0.8947, reflecting the additional training data available compared to the 100k subset (0.8643).

The OOD result (+4.99%, 0.6707 vs CNN 0.6208) confirms the finding from the 100k experimental search (Exp 21: +4.59%): graph-based structural reasoning transfers more robustly to born-digital documents than the CNN texture features learned from grayscale training data. Performance varies substantially across the 12 OOD classes: the model is strongest on letter (F1=0.853), scientific publication (F1=0.810), resume (F1=0.787), and news article (F1=0.766), which are classes with consistent structural layouts that transfer from grayscale training data. The weakest classes are specification (F1=0.340) and invoice (F1=0.561), both of which exhibit high structural variability in the born-digital domain, limiting the layout signal available to the graph.

The fusion architecture itself performed as designed: the model cannot degrade below a linear classifier on the CNN features, and the GNN embedding adds marginal structural signal on top. FusionGraphSAGE serves as the simplest validated GNN baseline for this task — demonstrating that graph-based reasoning is feasible at scale and that the

fusion shortcut is the minimum necessary architectural component. The GCN and DocumentGAT sections that follow explore alternative graph formulations that attempt to address the ceiling imposed by the 7×7 feature grid.

4.4 Graph Convolution Network (GCN)

While GraphSAGE and DocumentGAT formulate graphs internally within a single document image to capture local spatial layouts, Graph Convolutional Network (GCN) explores the opposite extreme: an inter-document transductive formulation. By treating the entire 320,000-image training split as a single, interconnected global graph, the GCN shifts the architectural focus from intra-document patch routing to cross-document visual consensus. This approach tests whether establishing neighborhood agreement among visually similar documents in the dataset manifold can provide robustness against severe distribution shifts.

Graph Features

The GCN discards the localized 7×7 spatial feature grid used in GraphSAGE. Instead, each node in the GCN represents an entire document, parameterized by the 2048-dimensional global average-pooled feature vector extracted from the fine-tuned ResNet-50 backbone.

Edges are constructed using L2-normalised feature-space cosine similarity. Because calculating a dense $320,000 \times 320,000$ similarity matrix exceeds standard GPU memory constraints, graph construction was dynamically batched in chunks of 10,000 nodes. For each document, directed edges are drawn to its

$K = 5$ nearest neighbors (with self-loops explicitly masked out), and the resulting graph is symmetrized. The choice of $k = 5$ was determined via ablation studies: sparser graphs force the GCN to rely only on the highest-confidence structural matches, whereas denser graphs ($k=10$ or 15) introduce noise by connecting structurally dissimilar documents, leading to over-smoothing.

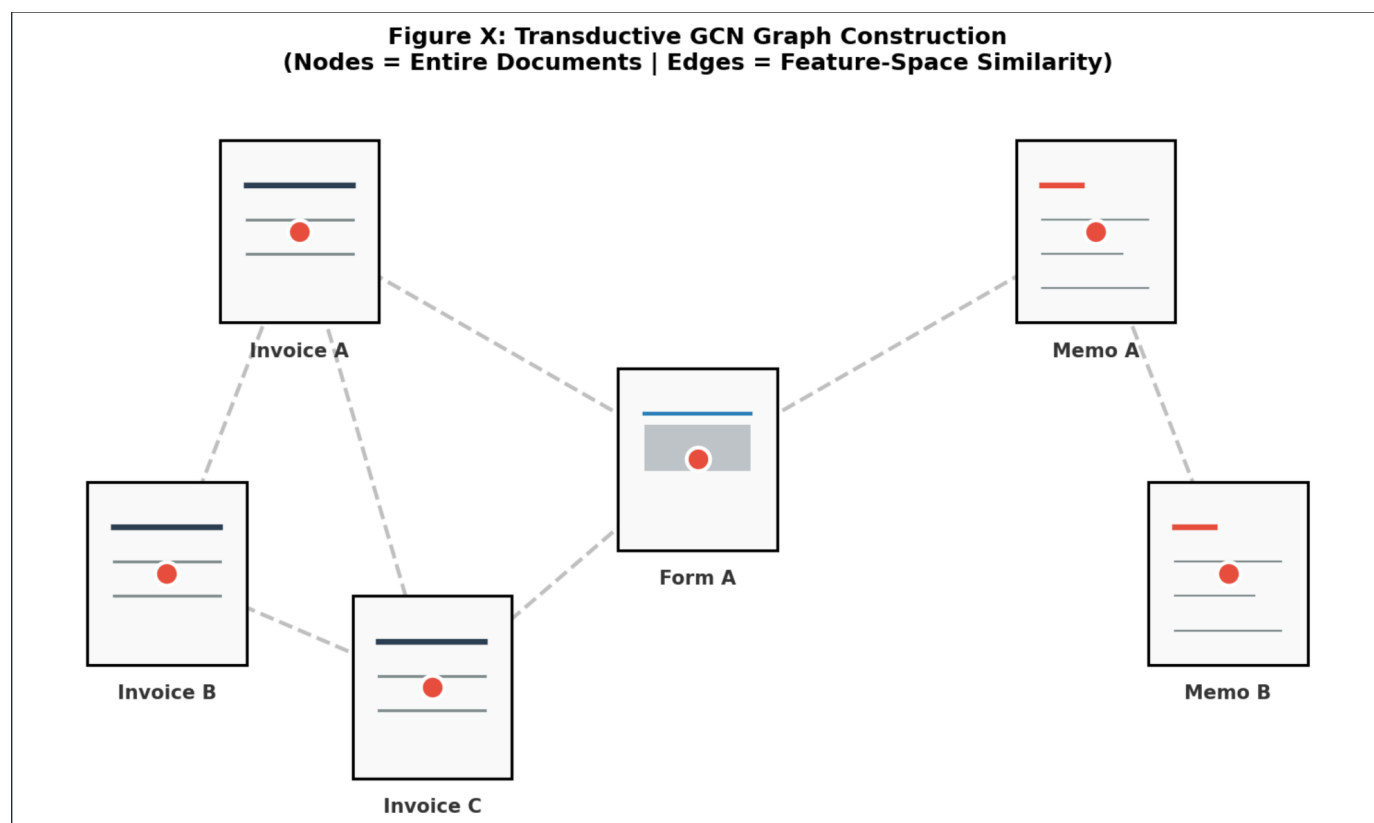


Figure 15: Transductive GCN Graph Construction. Unlike intra-document graphs, the GCN treats entire documents as single nodes, connecting them to their $k=5$ visually similar to nearest neighbors in the ResNet-50 feature space.

Model Architecture

The GCN processes the massive 320k-node graph through a three-layer message-passing architecture. Rather than projecting directly from 2048 dimensions to the 16 target classes—which creates a severe information bottleneck—the network employs a gradual compression strategy to provide sufficient representational capacity:

- Layer 1 ($2048 \rightarrow 512$): Aggregates high-dimensional neighbors features, followed by ReLU activation and dropout (0.3).
- Layer 2 ($512 \rightarrow 128$): Refines the node embeddings, followed by ReLU activation and dropout (0.3).
- Classification Head ($128 \rightarrow 16$): A final GCNConv layer mapping directly to the 16 document classes.

Crucially, unlike the GraphSAGE architecture, this GCN does not employ a fusion shortcut. By intentionally omitting the concatenation of the raw CNN features at the classification head, the network is forced to rely entirely on the aggregated neighborhood representations. Furthermore, the GCN performs node-level classification simultaneously across the entire batch, leveraging the full graph topology during a single forward pass.

Training Results

The GCN was trained using the Adam optimizer (learning rate = 1×10^{-3} , weight decay = 5×10^{-4}) for a maximum of 500 epochs. To ensure stable convergence and prevent overfitting on the dense transductive graph, early stopping was monitored via validation loss with a patience of 25 epochs.

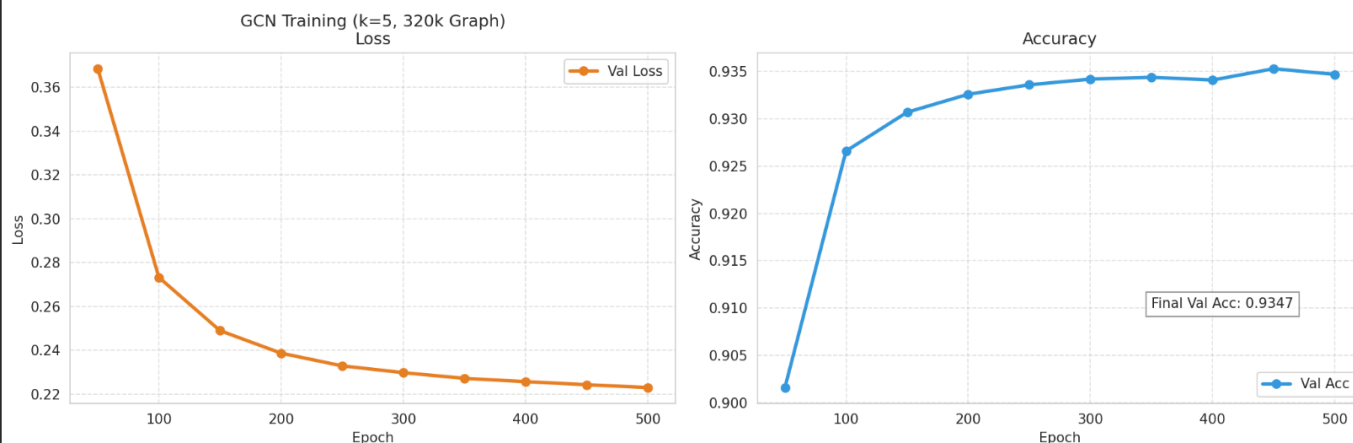
Figure Z: GCN Training Progression on the 320k Transductive Graph

Figure 16: GCN Training Progression on the 320k Transductive Graph. The model trains stably over 500 epochs, reaching 93.55% internal validation accuracy without triggering early stopping.

The model trained stably through all 500 epochs, achieving an internal validation accuracy of 0.9355 (validation loss: 0.2229). During formal inductive evaluation, the GCN achieved an in-distribution (ID) test accuracy of 0.8657 on the 40k RVL-CDIP test set. However, on the out-of-distribution (OOD) RVL-CDIP-N dataset (1,002 born-digital colour documents), the GCN achieved an accuracy of 0.7395 (macro-F1: 0.5214). This represents a massive +11.88 percentage point advantage over the CNN baseline (0.6208) on the same OOD data.

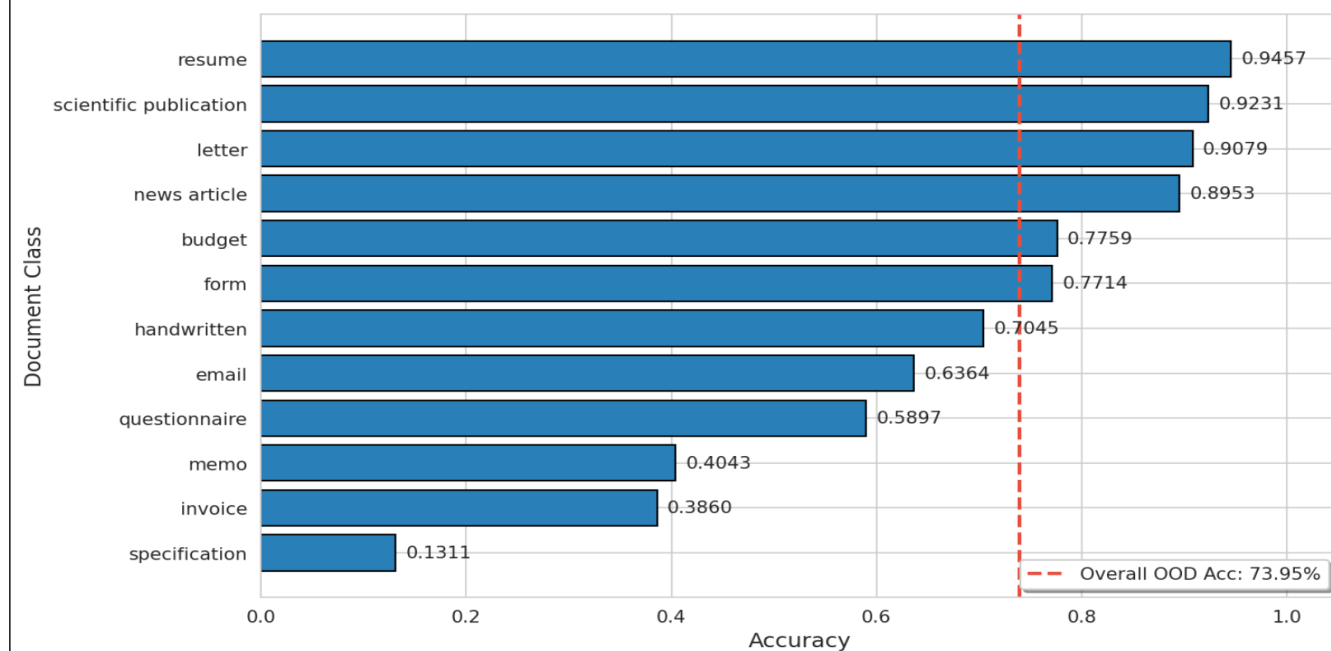
Figure Y: GCN Per-Class Accuracy on RVL-CDIP-N (Out-of-Distribution)

Figure 17: GCN Per-Class Accuracy on RVL-CDIP-N (Out-of-Distribution). The GCN demonstrates extreme robustness on structurally rigid classes like 'resume' and 'scientific publication' by leveraging cross-document consensus, despite severe domain shift.

Discussion

The GCN exhibits the most extreme behavioural divergence from the CNN baseline of all the models tested, perfectly illustrating the central trade-off of graph-based reasoning.

On clean, in-distribution data, the GCN suffers a 2.90 percentage point penalty relative to the CNN. This is primarily a result of transductive density mismatch and the intentional lack of a fusion shortcut. The GCN heavily optimises

its representations over the densely packed 320k training graph. At inference time, evaluating against held-out test nodes shifts the local graph density. Because the model lacks a direct shortcut to the highly discriminative fine-tuned CNN features, neighbourhood smoothing actively blurs representations that were already linearly separable in the baseline model.

However, this exact smoothing mechanism is the engine behind the GCN’s exceptional out-of-distribution performance (+11.88 pp). When evaluating on RVL-CDIP-N, the absolute photometric properties of the documents change drastically (e.g., from scanned grayscale to born-digital colour), heavily degrading individual CNN predictions. Yet, the relative visual similarity between documents of the same class remains largely intact. The GCN exploits this cross-image label consensus: by connecting an uncertain test document to its five most visually similar peers, the message-passing layers effectively execute a majority vote.

This dynamic is clearly visible in the per-class OOD breakdown (Figure Z). Structurally rigid classes with distinct macro-layouts—such as resume (0.9457), scientific publication (0.9231), and letter (0.9079)—survive the domain shift brilliantly, as their

kk-NN connections remain pure. Conversely, structurally heterogeneous classes like specification (0.1311) suffer catastrophic failure, as their lack of a consistent global layout causes the graph to draw noisy, cross-class edges, forcing the GCN to aggregate garbage information.

The primary limitation of this architecture lies in production deployment. Because the model's predictions rely entirely on cross-document edges, stateless, single-document REST API inference is not natively supported. Deployment of this GCN requires maintaining a cached reference corpus of node features and constructing a query-plus-corpus graph for every inference call, imposing a significant engineering overhead compared to the strictly inductive GraphSAGE and GAT pipelines.

4.5 Conclusion

Model	ID (40k test dataset)	Δ vs CNN	OOD (1002 test dataset)	Δ vs CNN
CNN baseline	0.8947	—	0.6208	—
DocumentGAT	0.8955	+0.08 pp	0.6577	+3.69 pp
FusionGraphSAGE	0.8847	-1.00 pp	0.6707	+4.99 pp
GCN	0.8657	-2.90 pp	0.7395	+11.88 pp

Table 4: Summary of GNN models tested against the ResNet-50 CNN benchmark

Part B evaluated three GNN architectures: (1) DocumentGAT, (2) FusionGraphSAGE, and (3)GCN against the fine-tuned ResNet-50 baseline on the RVL-CDIP test set and the RVL-CDIP-N out-of-distribution benchmark.

On in-distribution data, none of the three GNN variants significantly exceed the CNN baseline of 0.8947 on the 40k held-out test set. The consistent pattern is that graph-based reasoning adds no lift on clean RVL-CDIP documents. The fine-tuned ResNet-50 already encodes the regularities that drive document classification (header-body structure, tabular layout, textual density) and its 2048-dimensional output is close to linearly separable for the 16-class task. Rearranging that information through GNN message passing cannot recover signal the CNN has not already extracted, and in the GCN's case actively degrades accuracy by redistributing information across 2-hop neighbourhoods whose density is lower at inference than it was during training.

The picture inverts on out-of-distribution data. On RVL-CDIP-N the CNN baseline degrades sharply to 0.6208, a 27.4 pp drop from in-distribution, and every GNN variant beats it: DocumentGAT by +3.69 pp (0.6577), FusionGraphSAGE by +4.99 pp (0.6707), and the GCN by a substantially larger +11.88 pp (0.7395). The mechanism differs per architecture. DocumentGAT's gain comes from layout-derived edges (YOLO region centroids, DocLayNet classes, OCR text embeddings) that remain informative when pixel-level features degrade, since spatial arrangement is a more stable property of documents than photometric appearance. FusionGraphSAGE's gain comes from the concat fusion preserving the raw CNN signal as a lower bound, with the GraphSAGE embedding supplying additional context when the global CNN feature is noisy. The GCN's gain comes from cross-image label consensus: its k-NN graph connects each test document to its five most visually similar peers in the test batch, and its message-passing layers effectively take a majority vote across that neighbourhood.

The three architectures jointly reveal a clean trade-off: the GNN's in-distribution cost relative to the CNN is roughly proportional to its out-of-distribution gain. DocumentGAT sacrifices essentially nothing on ID (+0.08 pp) and earns +3.69 pp on OOD; FusionGraphSAGE gives up 1.00 pp on ID for +4.99 pp on OOD; and the GCN accepts a 2.90 pp ID penalty in exchange for +11.88 pp on OOD. The pattern suggests that the more a GNN's graph formulation departs from the CNN's native representation geometry, the less faithful it is to the CNN on clean data but the more robust it becomes under distribution shift, because the reshaping is structurally orthogonal to whatever the CNN encoded.

The trade-off is therefore better framed in deployment terms than in accuracy terms. DocumentGAT has 10x fewer trainable parameters than ResNet-50 but requires YOLO layout detection, Tesseract OCR, and a Sentence-BERT encoder at inference for a 3-4 pp OOD gain. FusionGraphSAGE is the simplest drop-in: a single GNN forward pass on the cached 7×7 feature map plus the pooled vector, no extra modalities or auxiliary models. The GCN delivers by far the largest OOD gain but only as a batch-inference system: single-document classification is not natively supported because the model's prediction depends on a graph over multiple documents. Production deployment of the GCN requires maintaining a reference corpus of representative features and constructing a query-plus-corpus graph per inference call, and accepts the 2.90 pp clean-data accuracy cost as the price of the +11.88 pp robustness margin.

The broader conclusion is that on a task with a well-tuned CNN backbone, graph-based reasoning does not supersede that backbone on in-distribution accuracy, but it consistently provides robustness gains under domain shift, with magnitude proportional to how structurally orthogonal the graph formulation is to the CNN's native representations. For the RVL-CDIP $\rightarrow$ RVL-CDIP-N shift specifically, cross-image label consensus (GCN) dominates over per-document structural reasoning (GAT, GraphSAGE), because the distribution shift preserves relative visual similarity while breaking absolute feature positions.

5. Document Classification Research App

This section documents how the trained document classifiers are packaged and deployed as an interactive research demo. The goal is to provide a reproducible end-to-end pipeline from raw image upload to predicted RVL-CDIP class, while remaining lightweight enough to run on a single workstation.

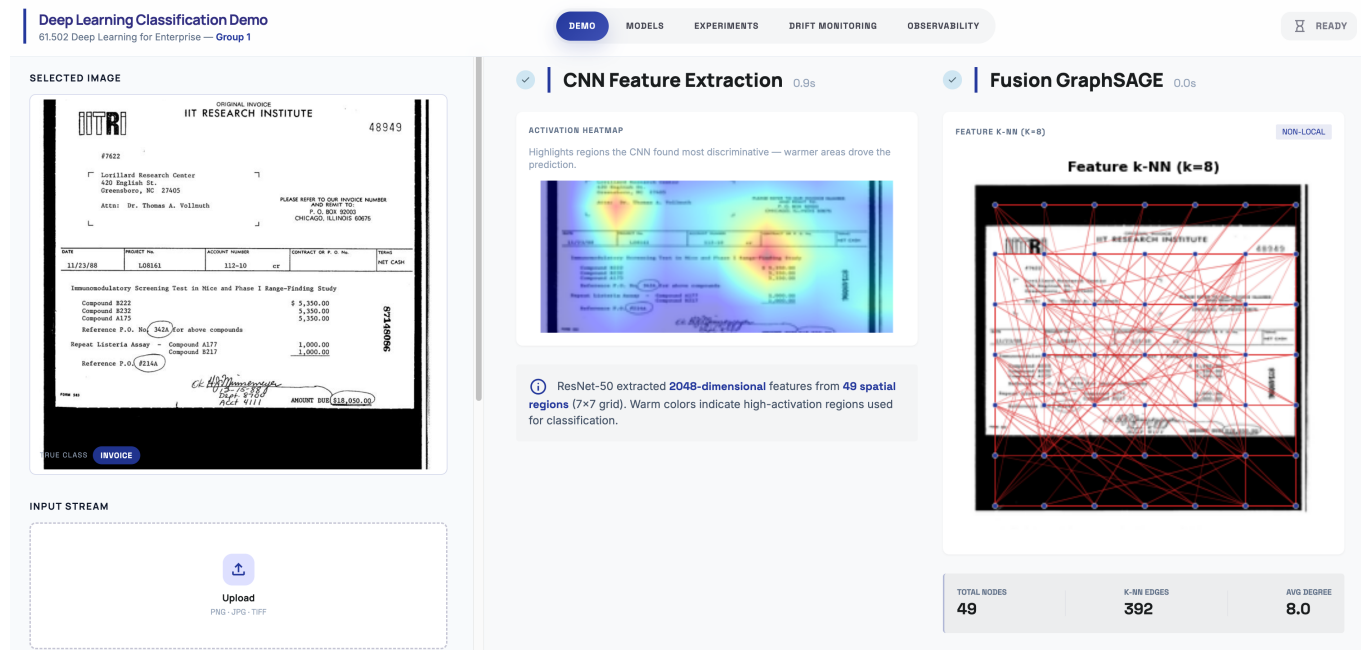


Figure 17: Doc Classification Research App showing feature construction for CNN and Fusion GraphSAGE for an invoice

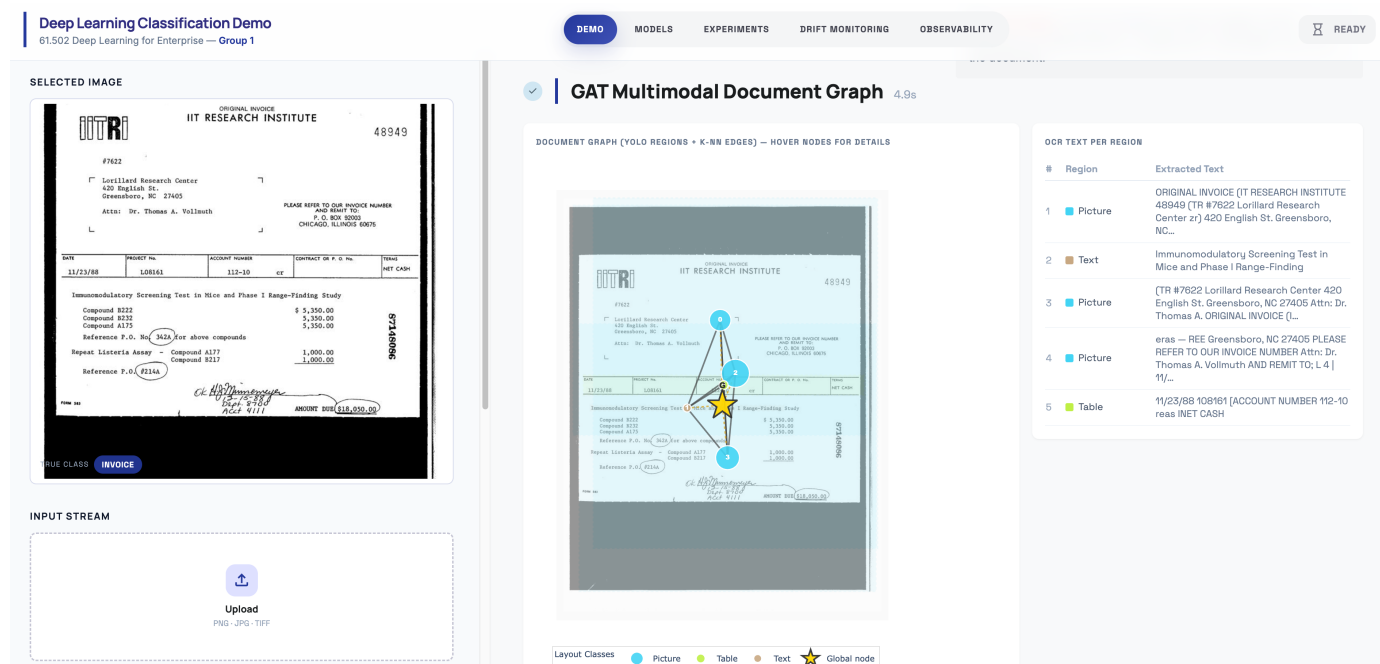


Figure 18: Document Classification Research App showing feature construction for Multimodal GAT for an invoice

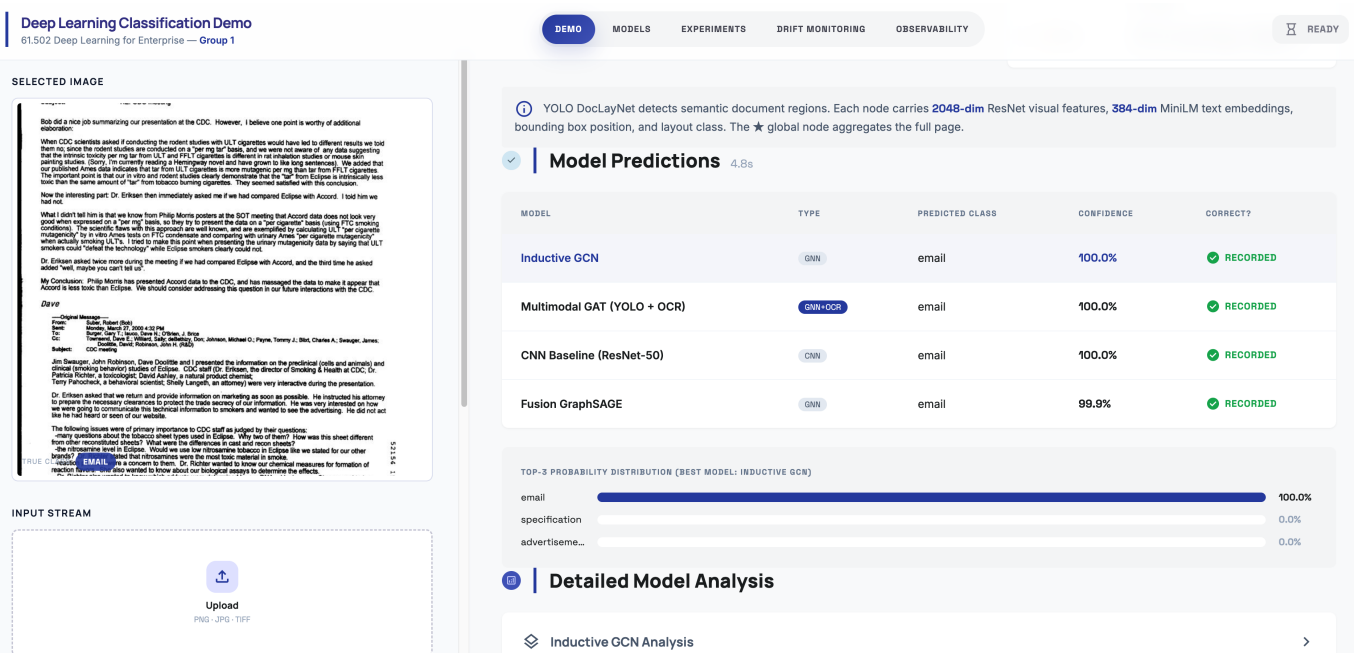


Figure 19: Doc Classification Research App showcasing model inference across 4 production models

5.1 Application Overview

The deployed application is a multi-model document classification research demo built on a FastAPI backend with a server-rendered Jinja2 and HTMX frontend. Users can upload a document image or select from curated sample sets—spanning in-distribution RVL-CDIP documents, out-of-distribution RVL-CDIP-N born-digital documents, and out-of-domain test cases—and the application runs inference across all four trained models simultaneously: the fine-tuned ResNet-50 CNN baseline, FusionGraphSAGE, Inductive GCN, and the Multimodal GAT. Each model returns its predicted RVL-CDIP class label, per-class confidence distribution, and intermediate representations such as the k-NN graph structure and CNN feature activations. The application is containerised using Docker Compose and exposes port 9000 for the inference service. A separate Evidently UI dashboard on port 8080 provides drift monitoring over the logged prediction stream, while a SQLite database records every inference event for downstream analysis. Structured application logs are shipped to a Seq log server, enabling real-time query and filtering of per-request inference telemetry across the full stack. The entire stack—model weights, application code, monitoring configuration, and log infrastructure—is version-controlled and reproducible from a single startup script

5.2 Source Code / Steps to Reproduce Models and Plots

All training, evaluation, and deployment code is maintained at <https://github.com/sagerstack/deep-learning-doc-classification>. The repository is organized as follows:

```
deep-learning-doc-classification/
├── .lab/                # Experiment tracking log and results TSV
├── .planning/           # Phase plans and verification docs
├── app/                 # FastAPI inference service + frontend
├── docs/                # Project report and reference materials
├── final-models/        # Production model weights (4 checkpoints)
├── final-notebooks/     # Definitive training and evaluation notebooks
├── monitoring/          # Evidently workspace, reference data, outputs
├── scripts/             # startup.sh, monitoring jobs, local dev
├── config.local-mini    # Experiment configs controlling dataset size,
├── config.local-10k     # device, batch size, and cache paths
├── config.local-100k    # (mini → 10k → 100k → full → cluster)
├── config.local-full
```

```

├── config.cluster
├── docker-compose.yml    # Container orchestration
├── Dockerfile            # App image definition
├── .env.example          # Environment variable template
├── pyproject.toml        # Python dependencies (Poetry)
├── poetry.lock           # Pinned dependency tree
└── README.md             # Setup and reproduction instructions

```

To reproduce all models and plots:

Step 1: Install Python dependencies	poetry install	
Step 2: Install the Tesseract OCR binary required by the Multimodal GAT notebook	brew install tesseract on macOS or apt install tesseract-ocr on Linux.	
Step 3: Run the notebooks in order from final-notebooks/	RVL-CDIP_ResNet50 1.ipynb	Fine-tunes ResNet-50 CNN baseline, generates accuracy curves and confusion matrix
	exp16_fusion_sage_results 1.ipynb	Trains FusionGraphSAGE, evaluates on ID and OOD test sets
	400K_GCN.ipynb	Trains Inductive GCN on the full 400k split
	RVL-CDIP_GAT_multimodal.ipynb	Trains Multimodal GAT with YOLO layout detection and Sentence-BERT text features

Note: RVL-CDIP_ResNet50 1.ipynb, 400K_GCN.ipynb, and RVL-CDIP_GAT_multimodal.ipynb are self-contained. exp16_fusion_sage_results 1.ipynb additionally requires src/model.py and src/train.py to be present at the project root.

5.3 Application Setup & Deployment Guide

The application stack is managed via a single startup script. All services run in Docker except the Evidently UI, which runs locally via Poetry.

Setup

Step	Action	Command / Detail
1	Clone repository and install dependencies	git clone https://github.com/sagerstack/deep-learning-doc-classification && poetry install
2	Configure environment	Copy .env.example → .env.local. Set APP_PORT (default: 9000).
3	Place model checkpoints	Add 4 .pt files to final-models/. The startup script validates all four are present before launch.
4	Start all services	scripts/startup.sh

Additional flags: --reset rebuilds the Docker image from scratch; --logs tails the app container logs after startup; --monitoring runs an Evidently drift snapshot on startup; --stop tears down all services cleanly.

Services

Service	URL	Purpose
Demo App	http://localhost:9000	FastAPI inference service
Evidently UI	http://localhost:8080	Drift monitoring dashboard
Seq	http://localhost:5341	Structured log server

5.4 Application Architecture

Figure X illustrates the end-to-end application architecture across four layers :

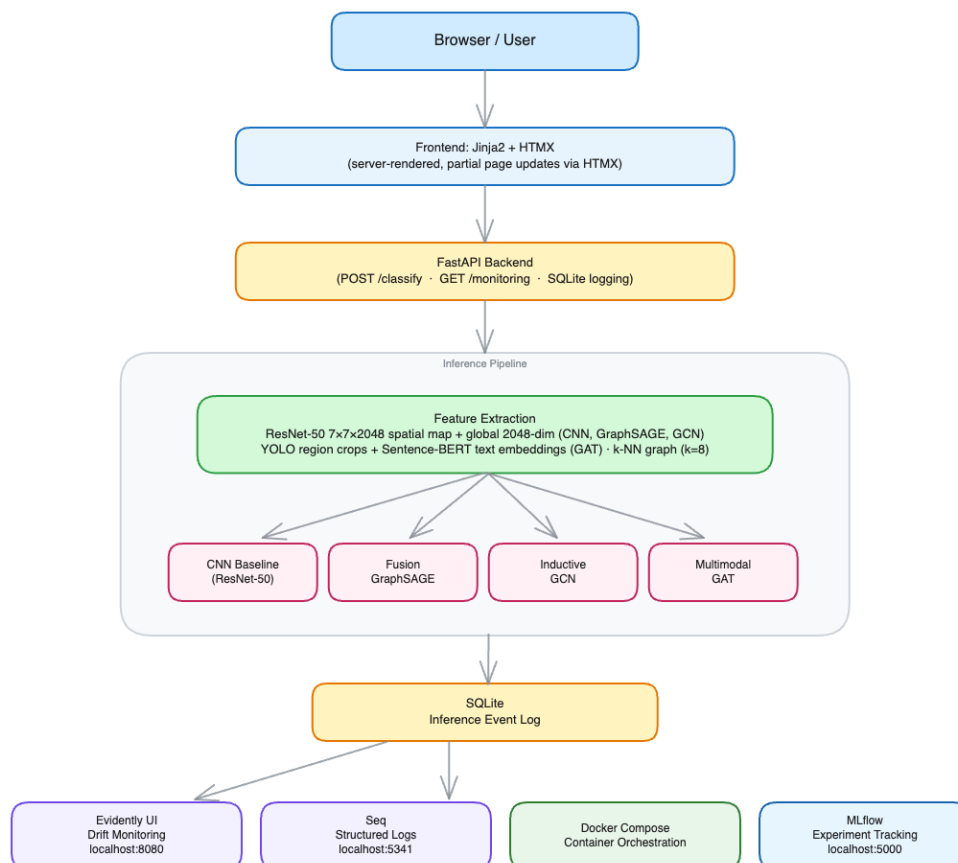


Figure 20: Architecture Diagram Doc Classification Research App

Frontend

The user interface is built with Jinja2 server-rendered templates and HTMX for partial page updates. When a user uploads an image or selects a sample document, HTMX submits the request and swaps only the results section of the page without a full reload, keeping the interaction lightweight with no JavaScript framework dependency.

API Layer

FastAPI handles all inference requests at POST /classify. The endpoint accepts an image upload or a sample selection, runs the full inference pipeline, logs the event to a SQLite database, and returns a rendered HTML partial for HTMX to inject into the page. A separate GET /monitoring route links out to the Evidently UI dashboard.

Inference Pipeline

Two feature extraction paths feed four models in parallel. The ResNet-50 backbone extracts a $7 \times 7 \times 2048$ spatial feature map and a 2048-dimensional global average-pooled vector; these feed the CNN Baseline, FusionGraphSAGE, and Inductive GCN. For the Multimodal GAT, YOLO detects layout regions within the document image, Tesseract extracts per-region text, and Sentence-BERT encodes it into semantic embeddings; a

feature-space k-NN graph ($k=8$, cosine similarity, 49 nodes) connects the regions before message passing. All four models return a predicted RVL-CDIP class label and per-class confidence distribution.

MLFlow

MLFlow tracks all training experiments and model metadata in a local SQLite-backed registry, accessible via the ‘Experiments’ tab on the main app.

Evidently

Evidently reads the inference log to generate classification and drift monitoring snapshots against a reference dataset, surfaced via the ‘Drift Monitoring’ tab on the main app.

Seq

Seq receives structured application logs via structlog/seqlog and exposes real-time application performance and model latency metrics. Accessed via the ‘Observability’ tab on the main app.

The entire stack is containerized with Docker Compose and managed by a single scripts/startup.sh entry point.

5.5 Development Methodology

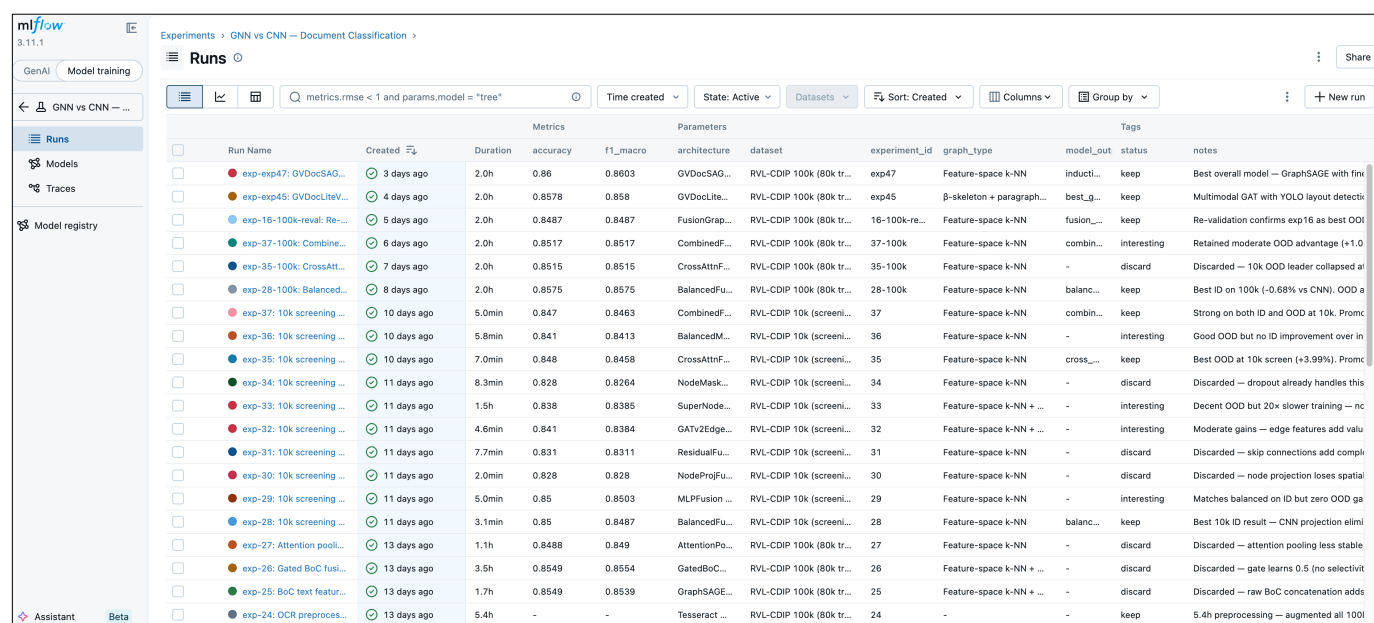
The following development practices were used throughout the course of project development, testing and deployment:

- **Methodology:** Iterative hypothesis-driven workflow — each experiment was logged in .lab/log.md with hyperparameters, results, and the decision made. 47 experiments across Phase A and B, progressing from baseline to ablation to final model selection.
- **SDLC:** Requirements (proposal) → design (graph construction strategy) → implementation (notebooks + app) → evaluation (ID/OOD metrics) → deployment (Docker + monitoring). Phases tracked via .planning/ GSD docs.
- **Dependencies:** All Python dependencies pinned in pyproject.toml and managed via Poetry. Docker ensures a reproducible runtime across machines.
- **Code quality:** PEP8 style, type hints throughout the app layer, modular service structure in app/src/services/. Git feature branches with PRs; no direct commits to main.
- **Testing:** Route and monitoring integration tests are implemented under app/tests/ using pytest, covering HTTP responses, HTMX partial rendering, SQLite event logging, and inference label feedback. Tests run against a real app instance (no mocking) via httpx
- **Version control:** Model checkpoints versioned by experiment ID in final-models/. Training configuration tracked per experiment in .lab/.

- **Infrastructure as Code:** The full deployment environment is defined in code — Dockerfile specifies the app image, docker-compose.yml orchestrates all services, pyproject.toml pins all Python dependencies, and scripts/startup.sh manages service lifecycle. No manual environment setup is required beyond placing model checkpoints.

5.6 Model Versioning

Experiment tracking and model versioning are managed with MLflow (v3.11.1). All 47 experiments conducted during the project are integrated with MLflow, providing a centralised view of experiment results and model lineage under the GNN vs CNN — Document Classification experiment. Each run records accuracy, macro F1, architecture, dataset, graph construction type, training duration, and a status tag (keep, discard, or interesting), along with a free-text notes field capturing the decision rationale. The MLflow Model Registry manages the four production checkpoints, enabling traceability from every experimental run through to the final deployed weights.



Run Name	Created	Duration	accuracy	f1_macro	architecture	dataset	experiment_id	graph_type	model_out	status	notes
exp-exp47: GVDocSAG...	3 days ago	2.0h	0.86	0.8603	GVDocSAG...	RVL-CDIP 100k (80k tr...	exp47	Feature-space k-NN	inducti...	keep	Best overall model — GraphSAGE with fin...
exp-exp45: GVDocLiteV...	4 days ago	2.0h	0.8578	0.858	GVDocLite...	RVL-CDIP 100k (80k tr...	exp45	β-skeleton + paragraph...	best_g...	keep	Multimodal GAT with YOLO layout detecti...
exp-16-100k-reval: Re...	5 days ago	2.0h	0.8487	0.8487	FusionGrap...	RVL-CDIP 100k (80k tr...	16-100k-re...	Feature-space k-NN	fusion_...	keep	Re-validation confirms exp16 as best OOI...
exp-37-100k: Combine...	6 days ago	2.0h	0.8517	0.8517	CombinedF...	RVL-CDIP 100k (80k tr...	37-100k	Feature-space k-NN	combin...	interesting	Retained moderate OOD advantage (+1.0...
exp-35-100k: CrossAtt...	7 days ago	2.0h	0.8515	0.8515	CrossAttF...	RVL-CDIP 100k (80k tr...	35-100k	Feature-space k-NN	-	discard	Discarded — 10k OOD leader collapsed at...
exp-28-100k: Balanced...	8 days ago	2.0h	0.8575	0.8575	BalancedFu...	RVL-CDIP 100k (80k tr...	28-100k	Feature-space k-NN	balanc...	keep	Best ID on 100k (-0.68% vs CNN). OOD a...
exp-37: 10k screening ...	10 days ago	5.0min	0.847	0.8463	CombinedF...	RVL-CDIP 10k (screeni...	37	Feature-space k-NN	combin...	keep	Strong on both ID and OOD at 10k. Promc...
exp-36: 10k screening ...	10 days ago	5.8min	0.841	0.8413	BalancedM...	RVL-CDIP 10k (screeni...	36	Feature-space k-NN	-	interesting	Good OOD but no ID improvement over in...
exp-35: 10k screening ...	10 days ago	7.0min	0.848	0.8458	CrossAttF...	RVL-CDIP 10k (screeni...	35	Feature-space k-NN	cross_...	keep	Best OOD at 10k screen (+3.99%). Promc...
exp-34: 10k screening ...	11 days ago	8.3min	0.828	0.8264	NodeMask...	RVL-CDIP 10k (screeni...	34	Feature-space k-NN	-	discard	Discarded — dropout already handles this...
exp-33: 10k screening ...	11 days ago	1.5h	0.838	0.8385	SuperNode...	RVL-CDIP 10k (screeni...	33	Feature-space k-NN + ...	-	interesting	Decent OOD but 20x slower training — nc...
exp-32: 10k screening ...	11 days ago	4.6min	0.841	0.8384	GATv2Edge...	RVL-CDIP 10k (screeni...	32	Feature-space k-NN + ...	-	interesting	Moderate gains — edge features add valu...
exp-31: 10k screening ...	11 days ago	7.7min	0.831	0.8311	ResidualFu...	RVL-CDIP 10k (screeni...	31	Feature-space k-NN	-	discard	Discarded — skip connections add compl...
exp-30: 10k screening ...	11 days ago	2.0min	0.828	0.828	NodeProFu...	RVL-CDIP 10k (screeni...	30	Feature-space k-NN	-	discard	Discarded — node projection loses spatial...
exp-29: 10k screening ...	11 days ago	5.0min	0.85	0.8503	MLPFusion ...	RVL-CDIP 10k (screeni...	29	Feature-space k-NN	-	interesting	Matches balanced on ID but zero OOD ga...
exp-28: 10k screening ...	11 days ago	3.1min	0.85	0.8487	BalancedFu...	RVL-CDIP 10k (screeni...	28	Feature-space k-NN	balanc...	keep	Best 10k ID result — CNN projection elimi...
exp-27: Attention pool...	13 days ago	1.1h	0.8488	0.849	AttentionPo...	RVL-CDIP 100k (80k tr...	27	Feature-space k-NN	-	discard	Discarded — attention pooling less stable...
exp-26: Gated BoC fusi...	13 days ago	3.5h	0.8549	0.8554	GatedBoC...	RVL-CDIP 100k (80k tr...	26	Feature-space k-NN + ...	-	discard	Discarded — gate learns 0.5 (no selectivi...
exp-25: BoC text featur...	13 days ago	1.7h	0.8549	0.8539	GraphSAGE...	RVL-CDIP 100k (80k tr...	25	Feature-space k-NN + ...	-	discard	Discarded — raw BoC concatenation adds...
exp-24: OCR preprocess...	13 days ago	5.4h	-	-	Tesseract...	RVL-CDIP 100k (80k tr...	24	-	-	keep	5.4h preprocessing — augmented all 100l...

Figure 21: MLflow dashboard for tracking experiments and ablation studies, and maintaining model registry

5.7 Model Performance Monitoring

Model performance monitoring is implemented using Evidently AI. A reference dataset of inference events is bootstrapped from the training distribution via scripts/monitoring/bootstrap_reference.py. On demand, scripts/monitoring/run_evidently.py reads the live inference event log from SQLite, computes classification metrics, data drift, and confidence drift against the reference, and pushes the results as snapshots into an Evidently UI workspace. A per-model dashboard tracks mean prediction confidence over time, the share and count of drifted input features, and prediction drift p-value. Drift trends are visualized across feature drift, confidence drift, and prediction drift dimensions, enabling monitoring of distribution shift between the training reference and live inference traffic without retraining.

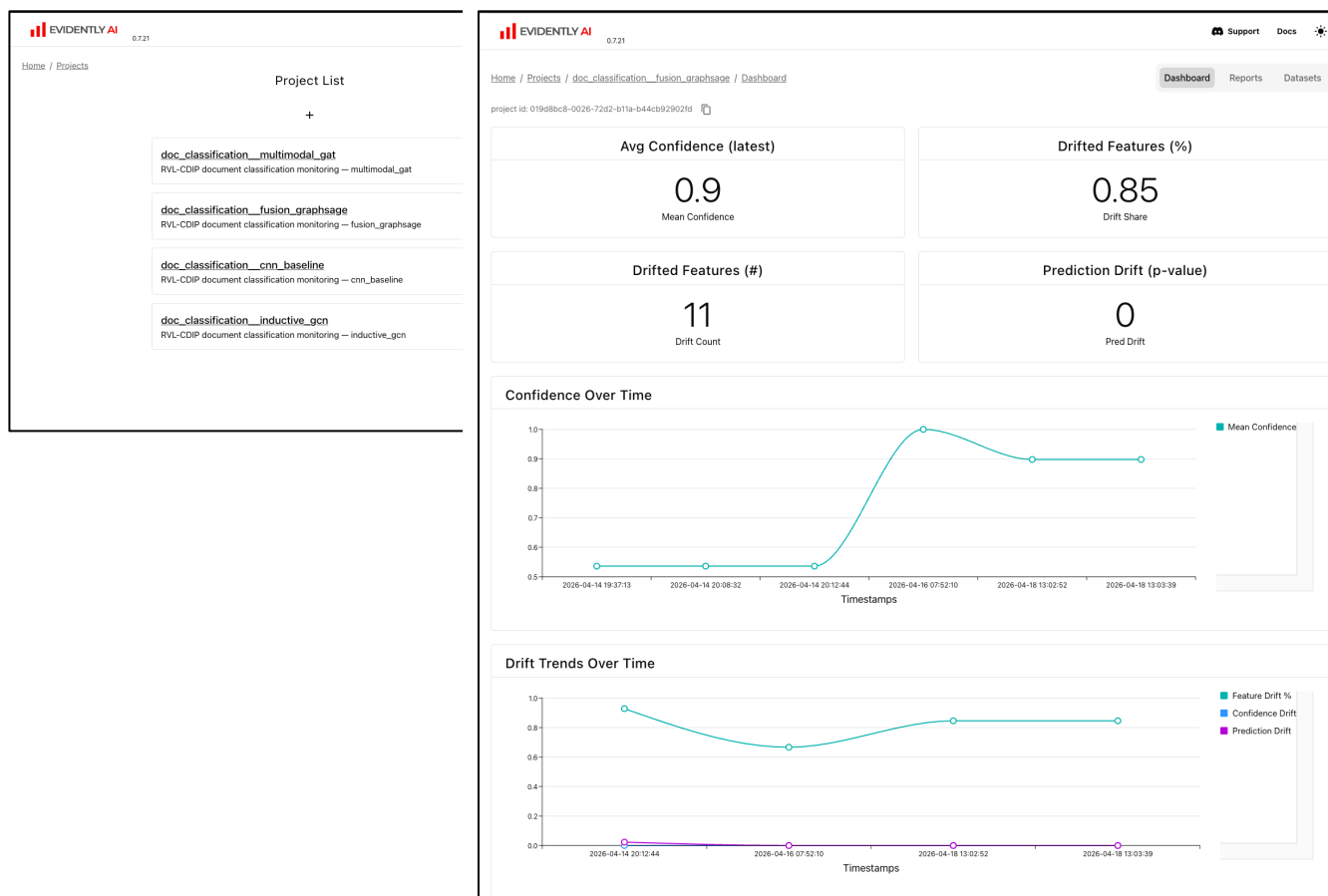


Figure 22: Evidently dashboard showing confidence and drift statistics across the four models deployed

5.8 Observability

Structured application logs are emitted via structlog and forwarded to Seq, where a custom Document Classification — Observability dashboard surfaces real-time telemetry across all four models. The dashboard tracks total requests, completions, failures, and average inference latency at both the aggregate and per-model level. Prediction distribution donut charts show per-class spread across all four models, and accuracy panels track live performance across three evaluation categories: in-distribution, out-of-distribution (RVL-CDIP-N), and out-of-domain — enabling immediate detection of performance degradation without querying the database.

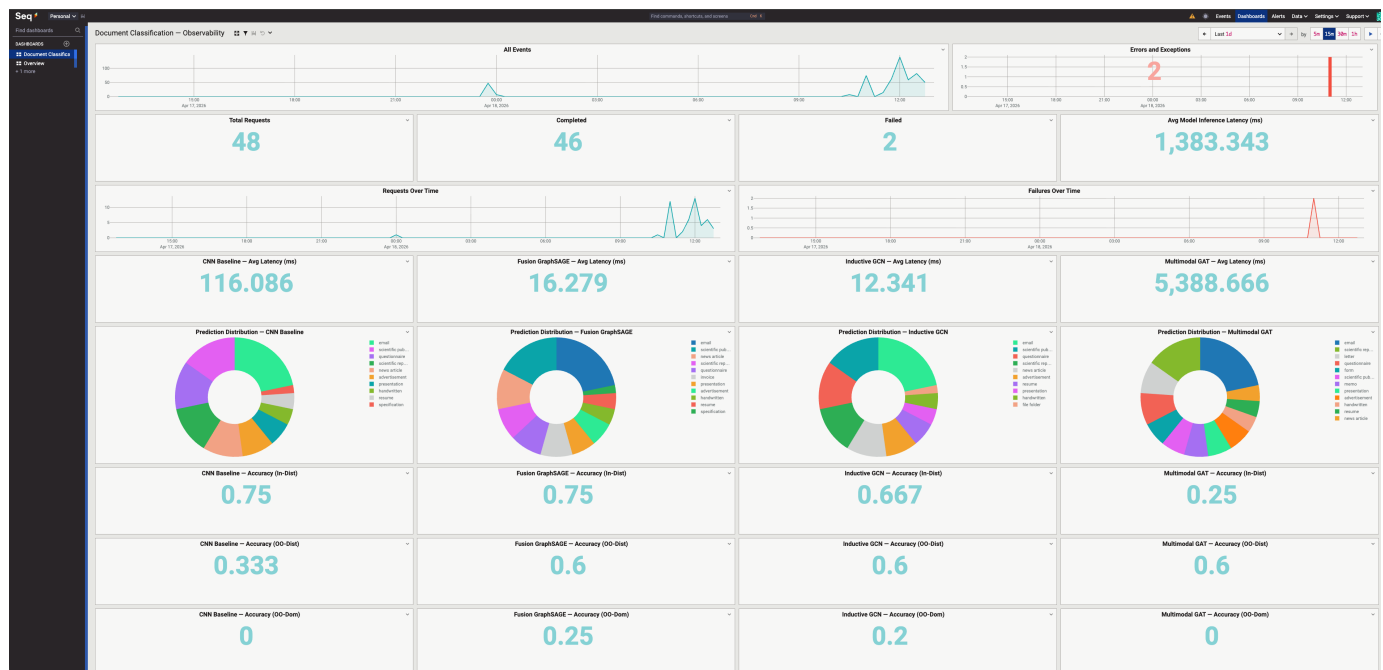


Figure 23: Seq dashboard for tracking app observability and model latency and performance metrics

6. Conclusion

Graph-based document classifiers offer a practical complement to large vision-language models: explicit spatial reasoning in an auditable structure, retrainable on proprietary document types without exposing data externally, and deployable on modest hardware without cloud API dependency. The consistent OOD generalisation advantage observed across all three GNN architectures, the central finding of this work, suggests that structural graph priors transfer across document distributions in ways not automatically guaranteed by scale alone. Future work should pursue OCR-derived semantic edges, stronger visual backbones, and multimodal feature fusion to close the accuracy gap with large models while retaining the cost, privacy, and auditability advantages that make graph-based document understanding viable in production.